PALMGUARD AI • SAWIT DETECTOR

Dokumentasi Kodingan Lengkap (Full Source Code)

Kumpulan kode sumber lengkap seluruh modul inti aplikasi Flutter deteksi penyakit pelepah kelapa sawit: UI, validasi citra pelepah sawit, mesin inferensi Forward Chaining, unit test, serta pipeline CI/CD GitHub Actions.

Nama Proyek	PalmGuard AI (`sawit_detector`)
Cakupan Dokumen	11 file sumber utama (3607 baris kode lengkap tanpa pemotongan)
Bahasa & SDK	Dart / Flutter (SDK `^3.11.5`) & YAML GitHub Actions
Repository GitHub	`https://github.com/Galaska77/sawit_detector`

Daftar File Kode Sumber Lengkap

Dokumen ini khusus memuat kode program lengkap (dari baris pertama hingga baris terakhir) untuk seluruh komponen utama aplikasi **PalmGuard AI**.

No	Path File	Komponen / Modul	Baris
1	`lib/main.dart`	Entry Point & Tema Aplikasi	30 baris
2	`lib/utils/forward_chaining.dart`	Basis Pengetahuan & Engine Forward Chaining	209 baris
3	`lib/utils/palm_image_analyzer.dart`	Validator Objek Pelepah Sawit & Analisis Citra Visual	648 baris
4	`lib/pages/home_page.dart`	Halaman Beranda (HomePage)	410 baris
5	`lib/pages/scan_page.dart`	Halaman Scan Kamera & Galeri (ScanPage)	804 baris
6	`lib/pages/result_page.dart`	Halaman Hasil Validasi & Diagnosis (ResultPage)	804 baris
7	`lib/pages/diagnosis_page.dart`	Halaman Kuisioner Gejala Manual (DiagnosisPage)	327 baris
8	`test/palm_image_analyzer_test.dart`	Unit Test Validasi Citra Pelepah Sawit	139 baris
9	`test/forward_chaining_test.dart`	Unit Test Engine Forward Chaining	66 baris
10	`.github/workflows/flutter_ci.yml`	Konfigurasi CI/CD GitHub Actions	72 baris
11	`pubspec.yaml`	Konfigurasi Proyek & Dependensi Flutter	98 baris
		Total Keseluruhan	**3607 baris**

File 1

Entry Point & Tema Aplikasi — `lib/main.dart`

> File utama yang menjalankan widget `PalmGuardApp`, mengatur tema gelap hijau sawit (`ColorScheme.dark`, warna latar `#0C1910`, aksen `#10B981`), dan mengarahkan layar awal ke `HomePage`.

lib/main.dart — Lengkap: Baris 1–30 (30 baris)

```
import 'package:flutter/material.dart';
import 'pages/home_page.dart';

void main() {
  runApp(const PalmGuardApp());
}

class PalmGuardApp extends StatelessWidget {
  const PalmGuardApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'PalmGuard AI',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        brightness: Brightness.dark,
        primaryColor: const Color(0xFF10B981),
        scaffoldBackgroundColor: const Color(0xFF0C1910),
        colorScheme: const ColorScheme.dark(
          primary: Color(0xFF10B981),
          secondary: Color(0xFF34D399),
          surface: Color(0xFF152A1C),
        ),
        fontFamily: 'Roboto',
      ),
      home: const HomePage(),
    );
  }
}
```

File 2

Basis Pengetahuan & Engine Forward Chaining — `lib/utils/forward_chaining.dart`

> Berisi model data `Symptom`, `Disease`, `Rule`, dan `DiagnosisResult`; daftar lengkap 13 gejala lapangan (`G01` – `G13`), 6 penyakit kelapa sawit (`P01` – `P06`), 6 aturan inferensi (`R1` – `R6`), serta fungsi `ForwardChainingEngine.diagnose()`.

lib/utils/forward_chaining.dart — **Lengkap: Baris 1–209 (209 baris)**

```
class Symptom {
  final String id;
  final String name;
  final String question;

  const Symptom({
    required this.id,
    required this.name,
    required this.question,
  });
}

class Disease {
  final String id;
  final String name;
  final String description;
  final String solution;

  const Disease({
    required this.id,
    required this.name,
    required this.description,
    required this.solution,
  });
}

class Rule {
  final String id;
  final List<String> symptomIds;
  final String diseaseId;

  const Rule({
    required this.id,
    required this.symptomIds,
    required this.diseaseId,
  });
}

class DiagnosisResult {
  final Disease disease;
  final double confidence;
  final List<String> matchedSymptoms;
  final int totalSymptoms;

  DiagnosisResult({
    required this.disease,
    required this.confidence,
    required this.matchedSymptoms,
    required this.totalSymptoms,
  });
}

class ForwardChainingEngine {
  static const List<Symptom> symptoms = [
    Symptom(
      id: 'G01',
      name: 'Pelepah daun tua menguning dan sengkleh (menggantung layu)',
      question: 'Apakah pelepah daun tua kelapa sawit terlihat menguning dan sengkleh (menggantung layu di sekitar
batang)?',
    ),
    Symptom(
      id: 'G02',
```

```
      name: 'Daun tombak (pucuk muda) tidak membuka lebih dari 2-3 pelepah',
      question: 'Apakah terdapat lebih dari 2-3 daun tombak (pucuk muda) yang tetap tegak dan tidak mau membuka?',
    ),
    Symptom(
      id: 'G03',
      name: 'Terdapat tubuh buah jamur (basidiokarp berbentuk setengah lingkaran) di pangkal batang',
      question: 'Apakah terlihat adanya tubuh buah jamur seperti kuping/setengah lingkaran keras berwarna coklat/putih di
pangkal batang?',
    ),
    Symptom(
      id: 'G04',
      name: 'Pangkal batang membusuk, berongga, atau mengeluarkan lendir kecoklatan',
      question: 'Apakah pangkal batang kelapa sawit membusuk, berongga, atau mengeluarkan cairan berlendir coklat saat
dibongkar?',
    ),
    Symptom(
      id: 'G05',
      name: 'Daun tombak (pucuk) membusuk, berwarna kecoklatan, dan sangat mudah dicabut',
      question: 'Apakah daun tombak di pucuk membusuk basah, berwarna coklat gelap, dan gampang sekali dicabut?',
    ),
    Symptom(
      id: 'G06',
      name: 'Keluar aroma busuk menyengat dari bagian titik tumbuh (pucuk sawit)',
      question: 'Apakah tercium aroma busuk menyengat (seperti pembusukan basah) dari area titik tumbuh di puncak batang?',
    ),
    Symptom(
      id: 'G07',
      name: 'Pelepah daun muda yang baru tumbuh membengkok, melengkung, atau terpelintir',
      question: 'Apakah pelepah daun muda yang baru keluar terlihat membengkok, melengkung abnormal, atau terpelintir
seperti mahkota?',
    ),
    Symptom(
      id: 'G08',
      name: 'Anak daun di bagian tengah pelepah tidak tumbuh/ompong (hanya lidi)',
      question: 'Apakah anak daun di bagian tengah pelepah tidak berkembang atau ompong, menyisakan lidi saja?',
    ),
    Symptom(
      id: 'G09',
      name: 'Serangan terjadi dominan pada tanaman muda umur 1 sampai 3 tahun',
      question: 'Apakah tanaman sawit yang terkena gejala ini tergolong masih muda (berumur sekitar 1-3 tahun)?',
    ),
    Symptom(
      id: 'G10',
      name: 'Pelepah daun di bagian tengah tajuk patah atau mengering terlebih dahulu',
      question: 'Apakah pelepah daun di bagian tengah tajuk/mahkota patah atau mengering duluan sebelum daun bagian
bawah?',
    ),
    Symptom(
      id: 'G11',
      name: 'Batang atas membusuk atau patah di ketinggian lebih dari 2 meter dari tanah',
      question: 'Apakah terlihat pembusukan batang atau batang patah di ketinggian lebih dari 2 meter dari permukaan
tanah?',
    ),
    Symptom(
      id: 'G12',
      name: 'Terdapat bercak coklat kehitaman dikelilingi warna kuning (halo) pada daun bibit',
      question: 'Apakah terdapat bercak-bercak coklat gelap kehitaman yang memiliki tepian lingkaran kuning terang pada
daun bibit?',
    ),
    Symptom(
      id: 'G13',
      name: 'Bercak kecil bulat kuning kecoklatan dengan pusat bercak agak cekung ke dalam',
      question: 'Apakah daun sawit dipenuhi bercak kecil bulat kuning kecoklatan dengan titik tengah yang agak cekung ke
dalam?',
    ),
  ];

  static const List<Disease> diseases = [
    Disease(
      id: 'P01',
      name: 'Busuk Pangkal Batang (Ganoderma)',
      description: 'Penyakit kelapa sawit paling mematikan yang disebabkan oleh jamur Ganoderma boninense. Jamur menyerang
dari dalam tanah dan membusukkan jaringan batang bagian bawah secara perlahan namun pasti hingga tanaman tumbang.',
```

```
      solution: '1. Lakukan pembubunan tanah di sekitar pangkal batang untuk memperkuat berdirinya tanaman.\n2. Aplikasikan fungisida berbahan aktif mikrobia antagonis seperti Trichoderma harzianum ke tanah sekitar pangkal batang.\n3. Jika tanaman sudah rusak parah (>50% pangkal batang busuk), segera bongkar tanaman, buat parit isolasi, dan bakar sisa tunggul agar tidak menular ke tanaman sehat sekitarnya.',
    ),
    Disease(
      id: 'P02',
      name: 'Busuk Pucuk (Bud Rot)',
      description: 'Kerusakan pada bagian pucuk tanaman sawit yang biasanya diawali oleh serangan bakteri Erwinia atau jamur Phytophthora setelah adanya luka fisik atau serangan hama kumbang tanduk. Jika meristem mati, tanaman tidak bisa tumbuh lagi.',
      solution: '1. Potong bagian pucuk yang membusuk sampai batas jaringan pelepah yang sehat.\n2. Semprotkan fungisida kontak dan sistemik dicampur bakterisida secara berkala di pucuk tanaman.\n3. Bersihkan sisa potongan dan taburkan kapur pertanian atau tembaga hidroksida untuk menekan kelembapan meristem.',
    ),
    Disease(
      id: 'P03',
      name: 'Penyakit Mahkota (Crown Disease)',
      description: 'Penyakit bawaan faktor genetik turunan yang biasanya menyerang kelapa sawit muda berumur 1-3 tahun. Ditandai dengan daun tengah bengkok terpelintir dan ompong.',
      solution: '1. Berikan pupuk makro dan mikro yang seimbang, terutama unsur Nitrogen dan Kalium secara teratur untuk mempercepat pemulihan tajuk daun baru.\n2. Lakukan kastrasi (pembuangan bunga/buah prematur) agar nutrisi fokus ke pemulihan vegetatif daun.\n3. Penyakit ini umumnya akan sembuh dengan sendirinya seiring bertambahnya umur tanaman (>3 tahun) apabila sifat bawaan genetiknya tidak terlalu parah.',
    ),
    Disease(
      id: 'P04',
      name: 'Busuk Batang Atas (Upper Stem Rot)',
      description: 'Pembusukan jaringan dalam batang kelapa sawit pada bagian tengah atau atas tajuk tanaman (>2 meter dari tanah). Disebabkan oleh infeksi spora jamur Ganoderma sp. yang menyebar melalui udara (aerogen) dan masuk lewat bekas luka pemanenan pelepah.',
      solution: '1. Bersihkan pelepah yang terinfeksi dan lakukan sanitasi bagian tajuk atas.\n2. Lakukan injeksi batang menggunakan fungisida sistemik berbahan aktif Hexaconazole atau Propioconazole.\n3. Hindari membuat luka mekanis yang berlebihan pada batang bagian atas saat proses pemanenan pelepah.',
    ),
    Disease(
      id: 'P05',
      name: 'Antraknosa (Anthracnose)',
      description: 'Penyakit bercak daun yang sangat merugikan di pembibitan kelapa sawit (nursery). Disebabkan oleh kelompok jamur Melanconium sp., Glomerella cingulata, atau Botryodiplodia sp. Menyebar sangat cepat melalui percikan air siraman.',
      solution: '1. Kurangi frekuensi penyiraman pembibitan di sore hari untuk menghindari daun basah dalam waktu lama di malam hari.\n2. Segera pisahkan (karantina) bibit yang menunjukkan gejala bercak agar tidak menyebar ke bibit sehat.\n3. Semprotkan fungisida protektif berbahan aktif Mancozeb atau Captan dengan dosis anjuran seminggu sekali.',
    ),
    Disease(
      id: 'P06',
      name: 'Bercak Daun Curvularia (Curvularia Leaf Spot)',
      description: 'Penyakit daun yang disebabkan oleh jamur Curvularia maculans. Sangat umum menyerang bibit muda di pre-nursery maupun main-nursery, ditandai bercak bulat kuning-coklat cekung.',
      solution: '1. Atur jarak antar polibag bibit (jangan terlalu rapat) guna memperlancar sirkulasi udara dan mengurangi kelembapan sekitar tajuk.\n2. Aplikasikan fungisida berbahan aktif Tembaga Hidroksida atau Difenokonazol secara berkala sesuai tingkat keparahan gejala.\n3. Berikan pemupukan nitrogen dan mikronutrien yang cukup agar imunitas sel daun bibit meningkat.',
    ),
  ];

  static const List<Rule> rules = [
    Rule(id: 'R1', symptomIds: ['G01', 'G02', 'G03', 'G04'], diseaseId: 'P01'),
    Rule(id: 'R2', symptomIds: ['G05', 'G06'], diseaseId: 'P02'),
    Rule(id: 'R3', symptomIds: ['G07', 'G08', 'G09'], diseaseId: 'P03'),
    Rule(id: 'R4', symptomIds: ['G10', 'G11'], diseaseId: 'P04'),
    Rule(id: 'R5', symptomIds: ['G12', 'G09'], diseaseId: 'P05'),
    Rule(id: 'R6', symptomIds: ['G13', 'G09'], diseaseId: 'P06'),
  ];

  /// Mendiagnosis penyakit kelapa sawit berdasarkan daftar symptomIds yang dipilih user.
  /// Menggunakan metode Forward Chaining.
  /// Mengembalikan list DiagnosisResult yang terurut berdasarkan confidence tertinggi.
  static List<DiagnosisResult> diagnose(List<String> selectedSymptomIds) {
    if (selectedSymptomIds.isEmpty) return [];

    List<DiagnosisResult> results = [];
```

```
    // Evaluasi setiap rule (Forward Chaining)
    for (var rule in rules) {
      List<String> matched = [];
      for (var sId in rule.symptomIds) {
        if (selectedSymptomIds.contains(sId)) {
          matched.add(sId);
        }
      }

      if (matched.isNotEmpty) {
        // Hitung persentase kecocokan (confidence)
        // confidence = (jumlah gejala yang cocok / total gejala pada rule) * 100
        double confidence = (matched.length / rule.symptomIds.length) * 100.0;

        // Dapatkan data Penyakit
        Disease disease = diseases.firstWhere((d) => d.id == rule.diseaseId);

        results.add(DiagnosisResult(
          disease: disease,
          confidence: confidence,
          matchedSymptoms: matched,
          totalSymptoms: rule.symptomIds.length,
        ));
      }
    }

    // Urutkan berdasarkan persentase kecocokan tertinggi
    results.sort((a, b) => b.confidence.compareTo(a.confidence));

    return results;
  }
}
```

File 3

Validator Objek Pelepah Sawit & Analisis Citra Visual — `lib/utils/palm_image_analyzer.dart`

> Implementasi lengkap pengolahan citra murni dengan `dart:ui` tanpa package CV eksternal. Mencakup dekode gambar (JPEG/PNG) ke piksel RGBA, konversi RGB ke HSV & Excess Green (ExG), klasifikasi piksel (daun hijau, kuning klorosis, coklat kering, busuk gelap, kulit/wajah, netral, non-tanaman), gerbang validasi untuk menolak objek selain pelepah sawit, deteksi kerapatan tepi tekstur lidi/daun, dan pemetaan gejala visual otomatis (`G01` , `G05` , `G10` , `G12` , `G13`).

lib/utils/palm_image_analyzer.dart — Lengkap: Baris 1–648 (648 baris)

```
import 'dart:math' as math;
import 'dart:typed_data';
import 'dart:ui' as ui;

/// Alasan mengapa gambar ditolak sebagai bukan pelepah/daun sawit.
enum PalmValidationError {
  invalidImage,
  tooDark,
  tooBright,
  nonPlantSkinOrFace,
  nonPlantColor,
  neutralBackground,
  noPalmFoliage,
  flatGreenSurface,
}

/// Hasil analisis citra pelepah kelapa sawit berbasis karakteristik visual.
class PalmImageAnalysis {
  final bool isPalmFrond;
  final PalmValidationError? validationError;
  final String statusTitle;
  final String statusDetail;
  final List<String> detectedSymptomIds;

  // Metrik proporsi visual (0.0 - 1.0)
  final double greenLeafRatio;
  final double yellowChlorosisRatio;
  final double brownDryRatio;
  final double darkRotRatio;
  final double skinOrNonPlantRatio;
  final double meanBrightness;
  final double edgeDensity;

  const PalmImageAnalysis({
    required this.isPalmFrond,
    this.validationError,
    required this.statusTitle,
    required this.statusDetail,
    required this.detectedSymptomIds,
    required this.greenLeafRatio,
    required this.yellowChlorosisRatio,
    required this.brownDryRatio,
    required this.darkRotRatio,
    required this.skinOrNonPlantRatio,
    required this.meanBrightness,
    required this.edgeDensity,
  });

  /// Mengembalikan persentase kesehatan daun relatif terhadap area vegetasi.
  double get healthyVegetationPercent {
    final total = greenLeafRatio + yellowChlorosisRatio + brownDryRatio + darkRotRatio;
    if (total <= 0) return 0.0;
    return (greenLeafRatio / total) * 100.0;
  }

  /// Mengembalikan persentase area menguning relatif terhadap area vegetasi.
  double get yellowingPercent {
    final total = greenLeafRatio + yellowChlorosisRatio + brownDryRatio + darkRotRatio;
    if (total <= 0) return 0.0;
```

```
    return (yellowChlorosisRatio / total) * 100.0;
  }

  /// Mengembalikan persentase bercak/busuk relatif terhadap area vegetasi.
  double get lesionPercent {
    final total = greenLeafRatio + yellowChlorosisRatio + brownDryRatio + darkRotRatio;
    if (total <= 0) return 0.0;
    return ((brownDryRatio + darkRotRatio) / total) * 100.0;
  }
}

enum _PixelClass {
  greenLeaf,
  yellowLeaf,
  dryBrown,
  darkRot,
  skinTone,
  nonPlantColor,
  neutral,
}

/// Penganalisis citra digital untuk memvalidasi apakah gambar merupakan pelepah/daun sawit
/// serta mengekstrak gejala visual penyakit secara langsung dari piksel gambar.
class PalmImageAnalyzer {
  static const int _targetGridSize = 96;

  /// Mendekode byte gambar terkompresi (JPEG/PNG) menggunakan `dart:ui` lalu menganalisisnya.
  static Future<PalmImageAnalysis> analyzeImageBytes(Uint8List encodedBytes) async {
    if (encodedBytes.isEmpty) {
      return _rejected(
        error: PalmValidationError.invalidImage,
        title: 'Gambar Tidak Valid',
        detail: 'Data gambar kosong atau tidak dapat dibaca.',
      );
    }

    ui.Codec? codec;
    ui.FrameInfo? frame;
    try {
      // Decode pada resolusi yang efisien agar cepat dan hemat memori
      codec = await ui.instantiateImageCodec(
        encodedBytes,
        targetWidth: 256,
      );
      frame = await codec.getNextFrame();
      final byteData = await frame.image.toByteData(format: ui.ImageByteFormat.rawRgba);
      if (byteData == null) {
        return _rejected(
          error: PalmValidationError.invalidImage,
          title: 'Gagal Membaca Piksel',
          detail: 'Format piksel gambar tidak didukung.',
        );
      }

      return analyzeRgba(
        byteData.buffer.asUint8List(),
        frame.image.width,
        frame.image.height,
      );
    } catch (e) {
      return _rejected(
        error: PalmValidationError.invalidImage,
        title: 'Gagal Memproses Gambar',
        detail: 'Terjadi kesalahan saat membaca gambar: $e',
      );
    } finally {
      frame?.image.dispose();
      codec?.dispose();
    }
  }

  /// Menganalisis buffer piksel RGBA mentah untuk memvalidasi pelepah sawit dan gejala visual.
  static PalmImageAnalysis analyzeRgba(Uint8List rgbaBytes, int width, int height) {
```

```
    if (width <= 0 || height <= 0 || rgbaBytes.length < width * height * 4) {
      return _rejected(
        error: PalmValidationError.invalidImage,
        title: 'Dimensi Gambar Tidak Valid',
        detail: 'Ukuran atau buffer gambar tidak sesuai.',
      );
    }

    final int gridW = math.min(width, _targetGridSize);
    final int gridH = math.min(height, _targetGridSize);
    final int totalSamples = gridW * gridH;

    final classes = List<_PixelClass>.filled(totalSamples, _PixelClass.neutral);
    final lums = Float64List(totalSamples);
    final exGs = Float64List(totalSamples);

    int greenCount = 0;
    int yellowCount = 0;
    int dryBrownCount = 0;
    int darkRotCount = 0;
    int skinCount = 0;
    int nonPlantColorCount = 0;
    int neutralCount = 0;
    int overexposedCount = 0;

    // Area tengah (ROI 60% x 60%) tempat bingkai kamera diarahkan
    int centerSamples = 0;
    int centerGreenCount = 0;
    int centerYellowCount = 0;
    int centerSkinCount = 0;
    int centerNonPlantCount = 0;

    double sumBrightness = 0.0;
    double sumExGOnGreen = 0.0;
    double sumExG2OnGreen = 0.0;

    for (int gy = 0; gy < gridH; gy++) {
      final int srcY = ((gy + 0.5) * height / gridH).floor().clamp(0, height - 1);
      final bool inCenterY = gy >= (gridH * 0.2) && gy <= (gridH * 0.8);

      for (int gx = 0; gx < gridW; gx++) {
        final int srcX = ((gx + 0.5) * width / gridW).floor().clamp(0, width - 1);
        final bool inCenter = inCenterY && gx >= (gridW * 0.2) && gx <= (gridW * 0.8);

        final int byteIdx = (srcY * width + srcX) * 4;
        final int r = rgbaBytes[byteIdx];
        final int g = rgbaBytes[byteIdx + 1];
        final int b = rgbaBytes[byteIdx + 2];

        final double rn = r / 255.0;
        final double gn = g / 255.0;
        final double bn = b / 255.0;

        final double maxC = math.max(rn, math.max(gn, bn));
        final double minC = math.min(rn, math.min(gn, bn));
        final double delta = maxC - minC;
        final double v = maxC;
        final double s = maxC == 0.0 ? 0.0 : delta / maxC;

        double h = 0.0;
        if (delta > 0.0) {
          if (maxC == rn) {
            h = 60.0 * (((gn - bn) / delta) % 6.0);
          } else if (maxC == gn) {
            h = 60.0 * (((bn - rn) / delta) + 2.0);
          } else {
            h = 60.0 * (((rn - gn) / delta) + 4.0);
          }
          if (h < 0.0) h += 360.0;
        }

        final double lum = 0.299 * rn + 0.587 * gn + 0.114 * bn;
        final double exG = 2.0 * gn - rn - bn;
```

```
        final int gIdx = gy * gridW + gx;
        lums[gIdx] = lum;
        exGs[gIdx] = exG;
        sumBrightness += v;

        if (v > 0.96 && s < 0.12) {
          overexposedCount++;
        }

        final _PixelClass pClass = _classifyPixel(r, g, b, rn, gn, bn, h, s, v, exG);
        classes[gIdx] = pClass;

        switch (pClass) {
          case _PixelClass.greenLeaf:
            greenCount++;
            sumExGOnGreen += exG;
            sumExG2OnGreen += exG * exG;
            if (inCenter) centerGreenCount++;
          case _PixelClass.yellowLeaf:
            yellowCount++;
            if (inCenter) centerYellowCount++;
          case _PixelClass.dryBrown:
            dryBrownCount++;
          case _PixelClass.darkRot:
            darkRotCount++;
          case _PixelClass.skinTone:
            skinCount++;
            if (inCenter) centerSkinCount++;
          case _PixelClass.nonPlantColor:
            nonPlantColorCount++;
            if (inCenter) centerNonPlantCount++;
          case _PixelClass.neutral:
            neutralCount++;
        }

        if (inCenter) centerSamples++;
      }
    }

    final double meanBrightness = sumBrightness / totalSamples;
    final double greenRatio = greenCount / totalSamples;
    final double yellowRatio = yellowCount / totalSamples;
    final double dryBrownRatio = dryBrownCount / totalSamples;
    final double darkRotRatio = darkRotCount / totalSamples;
    final double skinRatio = skinCount / totalSamples;
    final double nonPlantColorRatio = nonPlantColorCount / totalSamples;
    final double neutralRatio = neutralCount / totalSamples;
    final double overexposedRatio = overexposedCount / totalSamples;

    final double centerGreenRatio = centerSamples > 0 ? centerGreenCount / centerSamples : 0.0;
    final double centerYellowRatio = centerSamples > 0 ? centerYellowCount / centerSamples : 0.0;
    final double centerSkinRatio = centerSamples > 0 ? centerSkinCount / centerSamples : 0.0;
    final double centerNonPlantRatio = centerSamples > 0 ? centerNonPlantCount / centerSamples : 0.0;

    // Hitung kepadatan tepi / tekstur (untuk membedakan helai daun vs bidang polos)
    int edgePairs = 0;
    int totalPairs = 0;
    for (int gy = 0; gy < gridH - 1; gy++) {
      for (int gx = 0; gx < gridW - 1; gx++) {
        final int idx = gy * gridW + gx;
        final int rightIdx = idx + 1;
        final int downIdx = idx + gridW;

        totalPairs += 2;
        if ((lums[idx] - lums[rightIdx]).abs() > 0.06 ||
            (exGs[idx] - exGs[rightIdx]).abs() > 0.07) {
          edgePairs++;
        }
        if ((lums[idx] - lums[downIdx]).abs() > 0.06 ||
            (exGs[idx] - exGs[downIdx]).abs() > 0.07) {
          edgePairs++;
        }
      }
```

```
    }
    final double edgeDensity = totalPairs > 0 ? edgePairs / totalPairs : 0.0;

    double greenExGStdDev = 0.0;
    if (greenCount > 1) {
      final double meanExG = sumExGOnGreen / greenCount;
      final double variance = math.max(0.0, (sumExG2OnGreen / greenCount) - (meanExG * meanExG));
      greenExGStdDev = math.sqrt(variance);
    }

    final double combinedNonPlant = math.min(1.0, skinRatio + nonPlantColorRatio);

    // --- TAHAP 1: VALIDASI APAKAH OBJEK BENAR-BENAR PELEPAH / DAUN SAWIT ---
    if (meanBrightness < 0.13) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.tooDark,
        statusTitle: 'Pencahayaan Terlalu Gelap',
        statusDetail:
            'Gambar terlalu gelap untuk dianalisis. Pastikan pencahayaan cukup dan arahkan kamera langsung ke pelepah
sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    if (overexposedRatio > 0.72) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.tooBright,
        statusTitle: 'Gambar Terlalu Silau',
        statusDetail:
            'Cahaya terlalu terang sehingga warna dan tekstur daun sawit tidak dapat dikenali.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Tolak wajah / kulit manusia (terutama jika berada di tengah frame kamera)
    if ((centerSkinRatio >= 0.18 && centerGreenRatio < 0.15) ||
        ((centerSkinRatio >= 0.18 || skinRatio >= 0.25) && greenRatio < 0.18)) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.nonPlantSkinOrFace,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
            'Terdeteksi objek non-tanaman (seperti wajah/kulit atau objek berwarna krem/oranye). Fokuskan kamera hanya pada
pelepah atau helai daun kelapa sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Tolak warna non-vegetasi dominan (langit biru, baju merah/ungu, dll.)
    if ((nonPlantColorRatio >= 0.35 || centerNonPlantRatio >= 0.35) && greenRatio < 0.20) {
```

```
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.nonPlantColor,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
            'Warna dominan pada foto tidak sesuai dengan karakteristik vegetasi pelepah atau daun kelapa sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Tolak bidang netral/polos (tembok, meja abu-abu/putih, latar belakang ruangan)
    if (neutralRatio >= 0.72 && greenRatio < 0.12) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.neutralBackground,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
            'Gambar didominasi bidang netral atau polos (tembok, lantai, atau ruangan). Arahkan kamera lebih dekat ke
pelepah sawit.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Wajib memiliki komponen vegetasi hijau/daun sawit yang memadai di keseluruhan dan tengah frame
    final double totalFoliage = greenRatio + yellowRatio;
    final double centerFoliage = centerGreenRatio + centerYellowRatio;
    final bool hasEnoughPalmFoliage =
        (greenRatio >= 0.10 || (totalFoliage >= 0.28 && greenRatio >= 0.05 && skinRatio < 0.15)) &&
            centerFoliage >= 0.10;

    if (!hasEnoughPalmFoliage) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.noPalmFoliage,
        statusTitle: 'Pelepah Sawit Tidak Terdeteksi',
        statusDetail:
            'Objek di dalam area fokus tidak menunjukkan karakteristik daun/pelepah kelapa sawit. Pastikan pelepah berada
di tengah bingkai kamera.',
        detectedSymptomIds: const [],
        greenLeafRatio: greenRatio,
        yellowChlorosisRatio: yellowRatio,
        brownDryRatio: dryBrownRatio,
        darkRotRatio: darkRotRatio,
        skinOrNonPlantRatio: combinedNonPlant,
        meanBrightness: meanBrightness,
        edgeDensity: edgeDensity,
      );
    }

    // Tolak bidang hijau polos seragam tanpa tekstur daun/lidi (misal tembok cat hijau/layar hijau)
    if (edgeDensity < 0.015 &&
        greenExGStdDev < 0.035 &&
        yellowRatio < 0.02 &&
        darkRotRatio < 0.01) {
      return PalmImageAnalysis(
        isPalmFrond: false,
        validationError: PalmValidationError.flatGreenSurface,
        statusTitle: 'Bukan Pelepah Sawit',
        statusDetail:
```

```
          'Permukaan hijau terlihat polos dan seragam tanpa tekstur helai daun atau lidi pelepah sawit.',
      detectedSymptomIds: const [],
      greenLeafRatio: greenRatio,
      yellowChlorosisRatio: yellowRatio,
      brownDryRatio: dryBrownRatio,
      darkRotRatio: darkRotRatio,
      skinOrNonPlantRatio: combinedNonPlant,
      meanBrightness: meanBrightness,
      edgeDensity: edgeDensity,
    );
  }

  // --- TAHAP 2: EKSTRAKSI GEJALA PENYAKIT DARI CITRA PELEPAH SAWIT ---
  final int plantPixels = greenCount + yellowCount + dryBrownCount + darkRotCount;
  final double relGreen = plantPixels > 0 ? greenCount / plantPixels : 0.0;
  final double relYellow = plantPixels > 0 ? yellowCount / plantPixels : 0.0;
  final double relDryBrown = plantPixels > 0 ? dryBrownCount / plantPixels : 0.0;
  final double relDarkRot = plantPixels > 0 ? darkRotCount / plantPixels : 0.0;

  // Deteksi pola spasial: bercak gelap dengan halo kuning (Antraknosa) & bintik tersebar (Curvularia)
  int haloSpotPixels = 0;
  int scatteredSpotPixels = 0;
  for (int gy = 2; gy < gridH - 2; gy++) {
    for (int gx = 2; gx < gridW - 2; gx++) {
      final int idx = gy * gridW + gx;
      final _PixelClass c = classes[idx];

      if (c == _PixelClass.darkRot) {
        bool nearYellow = false;
        bool nearGreen = false;
        for (int dy = -4; dy <= 4; dy++) {
          final int ny = gy + dy;
          if (ny < 0 || ny >= gridH) continue;
          for (int dx = -4; dx <= 4; dx++) {
            final int nx = gx + dx;
            if (nx < 0 || nx >= gridW) continue;
            final _PixelClass nb = classes[ny * gridW + nx];
            if (nb == _PixelClass.yellowLeaf && dx.abs() <= 2 && dy.abs() <= 2) {
              nearYellow = true;
            }
            if (nb == _PixelClass.greenLeaf) {
              nearGreen = true;
            }
          }
        }
        if (nearYellow && nearGreen) {
          haloSpotPixels++;
        }
      }

      if (c == _PixelClass.yellowLeaf || c == _PixelClass.dryBrown) {
        int greenNeighbors = 0;
        for (int dy = -2; dy <= 2; dy++) {
          for (int dx = -2; dx <= 2; dx++) {
            if (dx == 0 && dy == 0) continue;
            if (classes[(gy + dy) * gridW + (gx + dx)] == _PixelClass.greenLeaf) {
              greenNeighbors++;
            }
          }
        }
        // Titik kecil yang dikelilingi dominan hijau menandakan bercak bintik terpisah
        if (greenNeighbors >= 14) {
          scatteredSpotPixels++;
        }
      }
    }
  }

  final double haloSpotRatio = plantPixels > 0 ? haloSpotPixels / plantPixels : 0.0;
  final double scatteredSpotRatio = plantPixels > 0 ? scatteredSpotPixels / plantPixels : 0.0;

  final List<String> detectedSymptoms = [];
```

```
    final bool isClearlyHealthy = relGreen >= 0.78 &&
        relYellow < 0.15 &&
        relDryBrown < 0.08 &&
        relDarkRot < 0.05 &&
        haloSpotRatio < 0.010 &&
        scatteredSpotRatio < 0.03;

    if (!isClearlyHealthy) {
      // G12: Bercak coklat kehitaman dikelilingi warna kuning (halo) -> Antraknosa (P05)
      if (haloSpotRatio >= 0.010 && relGreen >= 0.25) {
        detectedSymptoms.add('G12');
      }

      // G13: Bercak kecil bulat kuning kecoklatan tersebar -> Curvularia (P06)
      if (scatteredSpotRatio >= 0.03 && !detectedSymptoms.contains('G12')) {
        detectedSymptoms.add('G13');
      }

      // G05: Pembusukan basah berwarna coklat gelap -> Busuk Pucuk (P02)
      if (relDarkRot >= 0.16) {
        detectedSymptoms.add('G05');
      }

      // G01: Pelepah daun menguning luas (klorosis berat) -> indikasi visual Ganoderma (P01)
      if (relYellow >= 0.22) {
        detectedSymptoms.add('G01');
      }

      // G10: Pelepah mengering/coklat jerami luas -> indikasi Busuk Batang Atas (P04)
      if (relDryBrown >= 0.20) {
        detectedSymptoms.add('G10');
      }

      // Jika ada gangguan visual kombinasi ringan-sedang yang belum melewati threshold tunggal
      if (detectedSymptoms.isEmpty) {
        if (relDarkRot >= 0.08) {
          detectedSymptoms.add('G05');
        } else if (relYellow >= relDryBrown && relYellow >= 0.12) {
          detectedSymptoms.add('G01');
        } else if (relDryBrown >= 0.10) {
          detectedSymptoms.add('G10');
        } else if (scatteredSpotRatio >= 0.015) {
          detectedSymptoms.add('G13');
        }
      }
    }

    final String title = detectedSymptoms.isEmpty
        ? 'Pelepah Sawit Terdeteksi (Sehat)'
        : 'Pelepah Sawit Terdeteksi (Ada Gejala Visual)';

    final String detail = detectedSymptoms.isEmpty
        ? 'Daun dan pelepah didominasi warna hijau sehat tanpa indikasi klorosis, bercak halo, maupun pembusukan berarti.'
        : 'Analisis visual mendeteksi indikasi gejala pada pelepah (${detectedSymptoms.join(', ')}). Anda dapat melengkapi
gejala lapangan lain melalui kuisioner.';

    return PalmImageAnalysis(
      isPalmFrond: true,
      validationError: null,
      statusTitle: title,
      statusDetail: detail,
      detectedSymptomIds: detectedSymptoms,
      greenLeafRatio: greenRatio,
      yellowChlorosisRatio: yellowRatio,
      brownDryRatio: dryBrownRatio,
      darkRotRatio: darkRotRatio,
      skinOrNonPlantRatio: combinedNonPlant,
      meanBrightness: meanBrightness,
      edgeDensity: edgeDensity,
    );
  }

  static _PixelClass _classifyPixel(
```

```
    int r,
    int g,
    int b,
    double rn,
    double gn,
    double bn,
    double h,
    double s,
    double v,
    double exG,
  ) {
    if (s < 0.15) {
      return _PixelClass.neutral;
    }

    // Kulit manusia / wajah: merah dominan, tanpa excess green, saturasi sedang
    final int maxRgb = math.max(r, math.max(g, b));
    final int minRgb = math.min(r, math.min(g, b));
    final bool isSkinRgb = r > 85 &&
        g > 35 &&
        b > 15 &&
        (maxRgb - minRgb) > 15 &&
        (r - g).abs() > 12 &&
        r > g &&
        r > b;
    final bool isSkinHsv = ((h >= 0.0 && h <= 42.0) || h >= 335.0) &&
        s >= 0.12 &&
        s <= 0.72 &&
        v >= 0.20 &&
        v <= 0.96;
    if (isSkinRgb && isSkinHsv && exG < 0.0) {
      return _PixelClass.skinTone;
    }

    // Daun hijau sawit (spektrum hijau dengan excess green positif)
    if (h >= 62.0 && h <= 165.0 && s >= 0.16 && v >= 0.12 && v <= 0.95 && exG > 0.03) {
      return _PixelClass.greenLeaf;
    }

    // Daun menguning / klorosis
    if (h >= 40.0 && h < 62.0 && s >= 0.22 && v >= 0.22 && gn >= rn * 0.72 && gn > bn * 1.25) {
      return _PixelClass.yellowLeaf;
    }

    // Pembusukan / bercak coklat gelap kehitaman
    if (h >= 8.0 && h < 45.0 && s >= 0.18 && v >= 0.08 && v < 0.34 && r > b) {
      return _PixelClass.darkRot;
    }

    // Daun mengering / nekrosis coklat jerami
    if (h >= 18.0 && h < 45.0 && s >= 0.15 && s <= 0.75 && v >= 0.34 && v <= 0.85) {
      return _PixelClass.dryBrown;
    }

    // Warna non-vegetasi (biru, ungu, magenta, merah pekat)
    if (s >= 0.18 && ((h > 170.0 && h < 335.0) || (h < 18.0 && s > 0.38))) {
      return _PixelClass.nonPlantColor;
    }

    return _PixelClass.neutral;
  }

  static PalmImageAnalysis _rejected({
    required PalmValidationError error,
    required String title,
    required String detail,
  }) {
    return PalmImageAnalysis(
      isPalmFrond: false,
      validationError: error,
      statusTitle: title,
      statusDetail: detail,
      detectedSymptomIds: const [],
```

```
      greenLeafRatio: 0.0,
      yellowChlorosisRatio: 0.0,
      brownDryRatio: 0.0,
      darkRotRatio: 0.0,
      skinOrNonPlantRatio: 0.0,
      meanBrightness: 0.0,
      edgeDensity: 0.0,
    );
  }
}
```

File 4

Halaman Beranda (HomePage) — `lib/pages/home_page.dart`

> Antarmuka utama aplikasi yang memuat kartu aksi *Scan Kamera Instan* dan *Diagnosis Manual (Kuisioner)*, ringkasan statistik aturan & gejala, serta katalog interaktif 6 penyakit kelapa sawit menggunakan `ExpansionTile`.

lib/pages/home_page.dart — **Lengkap: Baris 1–410 (410 baris)**

```
import 'package:flutter/material.dart';
import '../utils/forward_chaining.dart';
import 'diagnosis_page.dart';
import 'scan_page.dart';

class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFF0C1910), // Deep Forest background
      body: SafeArea(
        child: SingleChildScrollView(
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 20.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                // Header Profile & Notification Icon
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Text(
                          'Halo, Planter!',
                          style: TextStyle(
                            color: Colors.grey[400],
                            fontSize: 14,
                            fontWeight: FontWeight.w500,
                          ),
                        ),
                        const SizedBox(height: 4),
                        const Text(
                          'PalmGuard AI',
                          style: TextStyle(
                            color: Colors.white,
                            fontSize: 24,
                            fontWeight: FontWeight.bold,
                            letterSpacing: 0.5,
                          ),
                        ),
                      ],
                    ),
                    Container(
                      padding: const EdgeInsets.all(10),
                      decoration: BoxDecoration(
                        color: const Color(0xFF1B3524),
                        borderRadius: BorderRadius.circular(12),
                        border: Border.all(color: const Color(0xFF2A4D35), width: 1),
                      ),
                      child: const Icon(
                        Icons.notifications_active_outlined,
                        color: Color(0xFF10B981),
                        size: 24,
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 28),
```

```
// Hero Banner (Glassmorphism effect)
Container(
  width: double.infinity,
  padding: const EdgeInsets.all(20),
  decoration: BoxDecoration(
    gradient: LinearGradient(
      colors: [
        const Color(0xFF1B3A26),
        const Color(0xFF0F2617).withValues(alpha: 0.9),
      ],
      begin: Alignment.topLeft,
      end: Alignment.bottomRight,
    ),
    borderRadius: BorderRadius.circular(20),
    border: Border.all(color: const Color(0xFF2B5637), width: 1.5),
    boxShadow: [
      BoxShadow(
        color: Colors.black.withValues(alpha: 0.3),
        blurRadius: 15,
        offset: const Offset(0, 8),
      ),
    ],
  ),
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Row(
        children: [
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
            decoration: BoxDecoration(
              color: const Color(0xFF10B981).withValues(alpha: 0.2),
              borderRadius: BorderRadius.circular(8),
              border: Border.all(color: const Color(0xFF10B981), width: 1),
            ),
            child: const Text(
              'FORWARD CHAINING',
              style: TextStyle(
                color: Color(0xFF10B981),
                fontSize: 10,
                fontWeight: FontWeight.bold,
                letterSpacing: 1,
              ),
            ),
          ),
          const Spacer(),
          const Row(
            children: [
              Icon(Icons.shield, color: Colors.amber, size: 16),
              SizedBox(width: 4),
              Text(
                'Akurasi 100%',
                style: TextStyle(
                  color: Colors.amber,
                  fontSize: 12,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          )
        ],
      ),
      const SizedBox(height: 16),
      const Text(
        'Deteksi Penyakit Sawit\nSecara Dini & Akurat',
        style: TextStyle(
          color: Colors.white,
          fontSize: 20,
          fontWeight: FontWeight.bold,
          height: 1.3,
        ),
      ),
```

```
                    const SizedBox(height: 8),
                    Text(
                      'Lindungi perkebunan Anda dari serangan Ganoderma dan penyakit berbahaya lainnya menggunakan
kecerdasan sistem pakar.',
                      style: TextStyle(
                        color: Colors.grey[350],
                        fontSize: 13,
                        height: 1.4,
                      ),
                    ),
                    const SizedBox(height: 20),
                    Column(
                      children: [
                        ElevatedButton.icon(
                          onPressed: () {
                            Navigator.push(
                              context,
                              MaterialPageRoute(
                                builder: (context) => const ScanPage(),
                              ),
                            );
                          },
                          icon: const Icon(Icons.photo_camera_outlined, size: 20),
                          label: const Text(
                            'Scan Kamera Instan (AI)',
                            style: TextStyle(fontSize: 15, fontWeight: FontWeight.bold),
                          ),
                          style: ElevatedButton.styleFrom(
                            backgroundColor: const Color(0xFF10B981),
                            foregroundColor: Colors.white,
                            minimumSize: const Size(double.infinity, 50),
                            shape: RoundedRectangleBorder(
                              borderRadius: BorderRadius.circular(12),
                            ),
                            elevation: 4,
                            shadowColor: const Color(0xFF10B981).withValues(alpha: 0.4),
                          ),
                        ),
                        const SizedBox(height: 12),
                        OutlinedButton.icon(
                          onPressed: () {
                            Navigator.push(
                              context,
                              MaterialPageRoute(
                                builder: (context) => const DiagnosisPage(),
                              ),
                            );
                          },
                          icon: const Icon(Icons.assignment_outlined, size: 18, color: Color(0xFF10B981)),
                          label: const Text(
                            'Diagnosis Manual (Kuisioner)',
                            style: TextStyle(color: Colors.white, fontSize: 14, fontWeight: FontWeight.w600),
                          ),
                          style: OutlinedButton.styleFrom(
                            side: const BorderSide(color: Color(0xFF10B981), width: 1.5),
                            minimumSize: const Size(double.infinity, 48),
                            shape: RoundedRectangleBorder(
                              borderRadius: BorderRadius.circular(12),
                            ),
                          ),
                        ),
                      ],
                    ),
                  ],
                ),
              ),
              const SizedBox(height: 28),

              // Info Cards Grid
              const Text(
                'Statistik Kebun & Sistem',
                style: TextStyle(
                  color: Colors.white,
```

```
                    fontSize: 18,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 14),
                Row(
                  children: [
                    Expanded(
                      child: _buildStatCard(
                        icon: Icons.grass,
                        title: 'Total Rule',
                        value: '${ForwardChainingEngine.rules.length} Aturan',
                        iconColor: const Color(0xFF10B981),
                      ),
                    ),
                    const SizedBox(width: 16),
                    Expanded(
                      child: _buildStatCard(
                        icon: Icons.bug_report,
                        title: 'Data Gejala',
                        value: '${ForwardChainingEngine.symptoms.length} Gejala',
                        iconColor: Colors.orangeAccent,
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 28),

                // Catalog of Diseases
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    const Text(
                      'Katalog Penyakit Sawit',
                      style: TextStyle(
                        color: Colors.white,
                        fontSize: 18,
                        fontWeight: FontWeight.bold,
                      ),
                    ),
                    Text(
                      'Detail info',
                      style: TextStyle(
                        color: Colors.grey[500],
                        fontSize: 13,
                      ),
                    ),
                  ],
                ),
                const SizedBox(height: 14),
                ListView.builder(
                  shrinkWrap: true,
                  physics: const NeverScrollableScrollPhysics(),
                  itemCount: ForwardChainingEngine.diseases.length,
                  itemBuilder: (context, index) {
                    final disease = ForwardChainingEngine.diseases[index];
                    return Padding(
                      padding: const EdgeInsets.only(bottom: 12),
                      child: Material(
                        color: const Color(0xFF152A1C),
                        shape: RoundedRectangleBorder(
                          borderRadius: BorderRadius.circular(14),
                          side: const BorderSide(color: Color(0xFF223E2A), width: 1),
                        ),
                        clipBehavior: Clip.antiAlias,
                        child: Theme(
                        data: Theme.of(context).copyWith(dividerColor: Colors.transparent),
                        child: ExpansionTile(
                          iconColor: const Color(0xFF10B981),
                          collapsedIconColor: Colors.grey[400],
                          title: Text(
                            disease.name,
                            style: const TextStyle(
```

```
                          color: Colors.white,
                          fontSize: 15,
                          fontWeight: FontWeight.bold,
                        ),
                      ),
                      subtitle: Text(
                        'ID: ${disease.id}',
                        style: TextStyle(
                          color: Colors.grey[500],
                          fontSize: 12,
                        ),
                      ),
                      children: [
                        Padding(
                          padding: const EdgeInsets.only(left: 16.0, right: 16.0, bottom: 16.0),
                          child: Column(
                            crossAxisAlignment: CrossAxisAlignment.start,
                            children: [
                              const Divider(color: Color(0xFF223E2A), height: 1),
                              const SizedBox(height: 12),
                              const Text(
                                'Deskripsi:',
                                style: TextStyle(
                                  color: Color(0xFF10B981),
                                  fontSize: 13,
                                  fontWeight: FontWeight.bold,
                                ),
                              ),
                              const SizedBox(height: 4),
                              Text(
                                disease.description,
                                style: TextStyle(
                                  color: Colors.grey[300],
                                  fontSize: 13,
                                  height: 1.4,
                                ),
                              ),
                              const SizedBox(height: 12),
                              const Text(
                                'Saran Penanganan:',
                                style: TextStyle(
                                  color: Colors.orangeAccent,
                                  fontSize: 13,
                                  fontWeight: FontWeight.bold,
                                ),
                              ),
                              const SizedBox(height: 4),
                              Text(
                                disease.solution,
                                style: TextStyle(
                                  color: Colors.grey[300],
                                  fontSize: 13,
                                  height: 1.4,
                                ),
                              ),
                            ],
                          ),
                        )
                      ],
                    ),
                  ),
                ),
              );
              },
            ),
          ],
        ),
      ),
    ),
  );
}
```

```
  Widget _buildStatCard({
    required IconData icon,
    required String title,
    required String value,
    required Color iconColor,
  }) {
    return Container(
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: const Color(0xFF152A1C),
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: const Color(0xFF223E2A), width: 1),
      ),
      child: Row(
        children: [
          Container(
            padding: const EdgeInsets.all(8),
            decoration: BoxDecoration(
              color: iconColor.withValues(alpha: 0.1),
              borderRadius: BorderRadius.circular(10),
            ),
            child: Icon(icon, color: iconColor, size: 24),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              mainAxisSize: MainAxisSize.min,
              children: [
                Text(
                  title,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: TextStyle(
                    color: Colors.grey[400],
                    fontSize: 12,
                  ),
                ),
                const SizedBox(height: 4),
                Text(
                  value,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: const TextStyle(
                    color: Colors.white,
                    fontSize: 15,
                    fontWeight: FontWeight.bold,
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    );
  }
}
```

File 5

Halaman Scan Kamera & Galeri (ScanPage) — `lib/pages/scan_page.dart`

> Mengelola siklus kamera (`camera`), kontrol lampu kilat, pemilih gambar dari galeri (`image_picker`), animasi garis laser pemindai, overlay progres analisis, serta pipeline pemanggilan `PalmImageAnalyzer.analyzeImageBytes()` dan `ForwardChainingEngine.diagnose()` menuju `ResultPage` .

lib/pages/scan_page.dart — Lengkap: Baris 1–804 (804 baris)

```
import 'dart:typed_data';
import 'package:camera/camera.dart';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
import '../utils/forward_chaining.dart';
import '../utils/palm_image_analyzer.dart';
import 'result_page.dart';

class ScanPage extends StatefulWidget {
  const ScanPage({super.key});

  @override
  State<ScanPage> createState() => _ScanPageState();
}

class _ScanPageState extends State<ScanPage>
    with SingleTickerProviderStateMixin, WidgetsBindingObserver {
  // Camera & gallery variables
  List<CameraDescription> _cameras = [];
  CameraController? _cameraController;
  final ImagePicker _imagePicker = ImagePicker();
  bool _isCameraInitialized = false;
  bool _hasCameraError = false;
  String _cameraErrorMessage = '';
  bool _isFlashOn = false;

  // UI state variables
  late AnimationController _scannerController;
  late Animation<double> _scannerAnimation;

  bool _isScanning = false;
  String _scanningStatus = 'Menyiapkan pemindai...';
  double _scanningProgress = 0.0;
  Uint8List? _capturedImageBytes;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _initializeCamera();

    // Setup laser scanning animation
    _scannerController = AnimationController(
      duration: const Duration(seconds: 2),
      vsync: this,
    )..repeat(reverse: true);

    _scannerAnimation = Tween<double>(begin: 0.0, end: 1.0).animate(
      CurvedAnimation(parent: _scannerController, curve: Curves.easeInOut),
    );
  }

  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    final controller = _cameraController;
    if (controller == null || !controller.value.isInitialized) {
      return;
    }

    if (state == AppLifecycleState.inactive) {
```

```
      controller.dispose();
      _cameraController = null;
      if (mounted) {
        setState(() {
          _isCameraInitialized = false;
          _isFlashOn = false;
        });
      }
    } else if (state == AppLifecycleState.resumed) {
      _initializeCamera();
    }
  }

  Future<void> _initializeCamera() async {
    if (mounted) {
      setState(() {
        _hasCameraError = false;
        _cameraErrorMessage = '';
      });
    }

    try {
      _cameras = await availableCameras();
      if (!mounted) return;

      if (_cameras.isNotEmpty) {
        final backCamera = _cameras.firstWhere(
          (camera) => camera.lensDirection == CameraLensDirection.back,
          orElse: () => _cameras.first,
        );

        await _cameraController?.dispose();
        final controller = CameraController(
          backCamera,
          ResolutionPreset.medium,
          enableAudio: false,
        );
        _cameraController = controller;

        await controller.initialize();
        if (!mounted) {
          await controller.dispose();
          return;
        }

        setState(() {
          _isCameraInitialized = true;
          _hasCameraError = false;
        });
      } else {
        setState(() {
          _isCameraInitialized = false;
          _hasCameraError = true;
          _cameraErrorMessage = 'Perangkat tidak memiliki modul kamera.';
        });
      }
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isCameraInitialized = false;
        _hasCameraError = true;
        _cameraErrorMessage =
            'Izin kamera ditolak atau gagal menginisialisasi kamera.';
      });
    }
  }

  Future<void> _toggleFlash() async {
    final controller = _cameraController;
    if (controller == null || !controller.value.isInitialized) return;

    try {
      final nextFlash = !_isFlashOn;
```

```
      await controller.setFlashMode(nextFlash ? FlashMode.torch : FlashMode.off);
      if (mounted) {
        setState(() {
          _isFlashOn = nextFlash;
        });
      }
    } catch (_) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Lampu kilat tidak tersedia pada kamera ini.')),
      );
    }
  }

  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    _cameraController?.dispose();
    _scannerController.dispose();
    super.dispose();
  }

  Future<void> _takePictureAndAnalyze() async {
    if (_isScanning) return;

    final controller = _cameraController;
    if (controller == null || !controller.value.isInitialized) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text(
            'Kamera belum aktif. Pastikan izin kamera diberikan lalu coba lagi.',
          ),
        ),
      );
      return;
    }

    try {
      setState(() {
        _isScanning = true;
        _scanningStatus = 'Mengambil gambar pelepah...';
        _scanningProgress = 0.1;
      });

      final XFile image = await controller.takePicture();
      final Uint8List bytes = await image.readAsBytes();

      if (_isFlashOn) {
        try {
          await controller.setFlashMode(FlashMode.off);
        } catch (_) {}
      }

      if (!mounted) return;
      await _runVisualAnalysisPipeline(bytes);
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isScanning = false;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Gagal mengambil gambar: $e')),
      );
    }
  }

  Future<void> _pickFromGalleryAndAnalyze() async {
    if (_isScanning) return;

    try {
      final XFile? picked = await _imagePicker.pickImage(
        source: ImageSource.gallery,
        imageQuality: 90,
```

```
      );
      if (picked == null || !mounted) return;

      final Uint8List bytes = await picked.readAsBytes();
      if (!mounted) return;

      if (_isFlashOn && _cameraController != null) {
        try {
          await _cameraController!.setFlashMode(FlashMode.off);
        } catch (_) {}
      }

      setState(() {
        _isScanning = true;
        _scanningStatus = 'Membaca gambar dari galeri...';
        _scanningProgress = 0.1;
      });

      await _runVisualAnalysisPipeline(bytes);
    } catch (e) {
      if (!mounted) return;
      setState(() {
        _isScanning = false;
      });
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Gagal memilih gambar dari galeri: $e')),
      );
    }
  }

  Future<void> _runVisualAnalysisPipeline(Uint8List imageBytes) async {
    setState(() {
      _capturedImageBytes = imageBytes;
      _isFlashOn = false;
      _scanningProgress = 0.2;
      _scanningStatus = 'Memvalidasi objek pelepah sawit...';
    });

    // Jalankan analisis citra nyata secara paralel dengan progres UI singkat
    final analysisFuture = PalmImageAnalyzer.analyzeImageBytes(imageBytes);

    final steps = <(double, String)>[
      (0.4, 'Mengecek spektrum vegetasi & tekstur daun...'),
      (0.65, 'Menganalisis klorosis kuning & bercak daun...'),
      (0.85, 'Mencocokkan gejala visual ke Forward Chaining...'),
    ];

    for (final (progress, status) in steps) {
      await Future<void>.delayed(const Duration(milliseconds: 350));
      if (!mounted) return;
      setState(() {
        _scanningProgress = progress;
        _scanningStatus = status;
      });
    }

    final PalmImageAnalysis analysis = await analysisFuture;
    if (!mounted) return;

    setState(() {
      _scanningProgress = 1.0;
      _scanningStatus = analysis.isPalmFrond
          ? 'Pelepah sawit terverifikasi. Menyiapkan hasil...'
          : 'Objek bukan pelepah sawit. Menyiapkan laporan...';
    });

    await Future<void>.delayed(const Duration(milliseconds: 250));
    if (!mounted) return;

    // Hanya jalankan Forward Chaining jika gambar benar-benar terdeteksi sebagai pelepah sawit
    final List<String> detectedSymptoms =
        analysis.isPalmFrond ? analysis.detectedSymptomIds : const [];
    final List<DiagnosisResult> diagnosisResults = analysis.isPalmFrond
```

```
        ? ForwardChainingEngine.diagnose(detectedSymptoms)
        : const [];

    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (context) => ResultPage(
          selectedSymptomIds: detectedSymptoms,
          results: diagnosisResults,
          imageBytes: imageBytes,
          visualAnalysis: analysis,
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: const Color(0xFF0C1910),
      appBar: AppBar(
        backgroundColor: const Color(0xFF0C1910),
        elevation: 0,
        leading: IconButton(
          icon: const Icon(Icons.arrow_back, color: Colors.white),
          onPressed: () => Navigator.pop(context),
        ),
        title: const Text(
          'Scan Pelepah Sawit',
          style: TextStyle(
            color: Colors.white,
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        centerTitle: true,
        actions: [
          if (_isCameraInitialized && !_isScanning)
            IconButton(
              onPressed: _toggleFlash,
              tooltip: _isFlashOn ? 'Matikan Lampu' : 'Nyalakan Lampu',
              icon: Icon(
                _isFlashOn ? Icons.flash_on : Icons.flash_off,
                color: _isFlashOn ? Colors.amber : Colors.white,
              ),
            ),
        ],
      ),
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              if (!_isScanning) ...[
                Row(
                  children: [
                    Container(
                      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 4),
                      decoration: BoxDecoration(
                        color: const Color(0xFF10B981).withValues(alpha: 0.15),
                        borderRadius: BorderRadius.circular(8),
                        border: Border.all(color: const Color(0xFF10B981)),
                      ),
                      child: const Text(
                        'FOKUS: PELEPAH SAWIT',
                        style: TextStyle(
                          color: Color(0xFF10B981),
                          fontSize: 10,
                          fontWeight: FontWeight.bold,
                          letterSpacing: 1,
                        ),
                      ),
```

```
                    ),
                  ],
                ),
                const SizedBox(height: 10),
                const Text(
                  'Arahkan Kamera ke Pelepah Sawit',
                  style: TextStyle(
                    color: Colors.white,
                    fontSize: 20,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 6),
                Text(
                  'Arahkan kamera atau pilih gambar dari galeri khusus objek pelepah/daun sawit. Objek selain sawit (wajah, tembok, benda lain) akan otomatis ditolak.',
                  style: TextStyle(
                    color: Colors.grey[400],
                    fontSize: 12,
                    height: 1.4,
                  ),
                ),
                const SizedBox(height: 16),

                // Viewfinder / Camera Preview
                Expanded(
                  child: ClipRRect(
                    borderRadius: BorderRadius.circular(20),
                    child: Container(
                      width: double.infinity,
                      decoration: BoxDecoration(
                        color: const Color(0xFF102015),
                        border: Border.all(color: const Color(0xFF10B981), width: 1.5),
                        borderRadius: BorderRadius.circular(20),
                      ),
                      child: LayoutBuilder(
                        builder: (context, constraints) {
                          return Stack(
                            children: [
                              if (_isCameraInitialized && _cameraController != null)
                                Positioned.fill(
                                  child: FittedBox(
                                    fit: BoxFit.cover,
                                    clipBehavior: Clip.hardEdge,
                                    child: SizedBox(
                                      width: constraints.maxWidth,
                                      child: CameraPreview(_cameraController!),
                                    ),
                                  ),
                                )
                              else
                                Positioned.fill(
                                  child: Container(
                                    color: const Color(0xFF0F2417),
                                    padding: const EdgeInsets.all(24),
                                    child: Column(
                                      mainAxisAlignment: MainAxisAlignment.center,
                                      children: [
                                        Icon(
                                          _hasCameraError
                                              ? Icons.videocam_off_outlined
                                              : Icons.camera_alt_outlined,
                                          color: Colors.grey[400],
                                          size: 52,
                                        ),
                                        const SizedBox(height: 12),
                                        Text(
                                          _hasCameraError
                                              ? _cameraErrorMessage
                                              : 'Menginisialisasi kamera...',
                                          textAlign: TextAlign.center,
                                          style: TextStyle(
                                            color: Colors.grey[300],
```

```
                fontSize: 13,
              ),
            ),
            if (_hasCameraError) ...[
              const SizedBox(height: 16),
              OutlinedButton.icon(
                onPressed: _initializeCamera,
                icon: const Icon(
                  Icons.refresh,
                  size: 18,
                  color: Color(0xFF10B981),
                ),
                label: const Text(
                  'Coba Aktifkan Kamera',
                  style: TextStyle(
                    color: Color(0xFF10B981),
                    fontWeight: FontWeight.bold,
                  ),
                ),
                style: OutlinedButton.styleFrom(
                  side: const BorderSide(
                    color: Color(0xFF10B981),
                  ),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(10),
                  ),
                ),
              ),
            ],
          ],
        ),
      ),
    ),

// Corner Markers
_buildCornerMarkers(),

// Label area tengah pemindaian
Positioned(
  bottom: 16,
  left: 16,
  right: 16,
  child: Center(
    child: Container(
      padding: const EdgeInsets.symmetric(
        horizontal: 12,
        vertical: 6,
      ),
      decoration: BoxDecoration(
        color: Colors.black.withValues(alpha: 0.6),
        borderRadius: BorderRadius.circular(20),
        border: Border.all(
          color: const Color(0xFF10B981).withValues(alpha: 0.6),
        ),
      ),
      child: const Text(
        'Posisikan pelepah sawit di tengah bingkai',
        style: TextStyle(
          color: Colors.white,
          fontSize: 11,
          fontWeight: FontWeight.w500,
        ),
      ),
    ),
  ),
),

// Responsive scanner laser line
AnimatedBuilder(
  animation: _scannerAnimation,
  builder: (context, child) {
    final maxTop =
        (constraints.maxHeight - 4).clamp(0.0, double.infinity);
```

```
                                return Positioned(
                                  top: _scannerAnimation.value * maxTop,
                                  left: 0,
                                  right: 0,
                                  child: Container(
                                    height: 3,
                                    decoration: BoxDecoration(
                                      color: const Color(0xFF10B981),
                                      boxShadow: [
                                        BoxShadow(
                                          color: const Color(0xFF10B981)
                                              .withValues(alpha: 0.6),
                                          blurRadius: 8,
                                          spreadRadius: 2,
                                        ),
                                      ],
                                    ),
                                  ),
                                );
                              },
                            ),
                          ],
                        );
                      },
                    ),
                  ),
                ),
                const SizedBox(height: 16),

                // Panduan singkat akurasi visual
                Container(
                  width: double.infinity,
                  padding: const EdgeInsets.all(14),
                  decoration: BoxDecoration(
                    color: const Color(0xFF152A1C),
                    borderRadius: BorderRadius.circular(14),
                    border: Border.all(color: const Color(0xFF223E2A)),
                  ),
                  child: Row(
                    children: [
                      const Icon(
                        Icons.eco_outlined,
                        color: Color(0xFF10B981),
                        size: 22,
                      ),
                      const SizedBox(width: 12),
                      Expanded(
                        child: Text(
                          'Deteksi warna daun hijau, klorosis kuning, bercak halo, dan nekrosis coklat langsung dari
gambar.',
                          style: TextStyle(
                            color: Colors.grey[300],
                            fontSize: 12,
                            height: 1.35,
                          ),
                        ),
                      ),
                    ],
                  ),
                ),
                const SizedBox(height: 16),

                // Action Buttons: Galeri & Kamera
                Row(
                  children: [
                    Expanded(
                      flex: 2,
                      child: OutlinedButton.icon(
                        onPressed: _pickFromGalleryAndAnalyze,
                        icon: const Icon(
                          Icons.photo_library_outlined,
                          color: Color(0xFF10B981),
```

```
                    size: 20,
                  ),
                  label: const Text(
                    'Galeri',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 14,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  style: OutlinedButton.styleFrom(
                    side: const BorderSide(color: Color(0xFF10B981), width: 1.5),
                    minimumSize: const Size(double.infinity, 54),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
              ),
              const SizedBox(width: 12),
              Expanded(
                flex: 3,
                child: ElevatedButton.icon(
                  onPressed: _isCameraInitialized ? _takePictureAndAnalyze : null,
                  icon: const Icon(Icons.photo_camera),
                  label: const Text(
                    'Ambil Foto',
                    style: TextStyle(fontSize: 15, fontWeight: FontWeight.bold),
                  ),
                  style: ElevatedButton.styleFrom(
                    backgroundColor: const Color(0xFF10B981),
                    disabledBackgroundColor: const Color(0xFF1B3524),
                    disabledForegroundColor: Colors.grey[500],
                    foregroundColor: Colors.white,
                    minimumSize: const Size(double.infinity, 54),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                    elevation: 5,
                    shadowColor: const Color(0xFF10B981).withValues(alpha: 0.3),
                  ),
                ),
              ),
            ),
          ],
        ),
      ] else ...[
        // SCANNING ACTIVE STATE
        Expanded(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Center(
                child: ClipRRect(
                  borderRadius: BorderRadius.circular(20),
                  child: Container(
                    width: double.infinity,
                    height: 300,
                    decoration: BoxDecoration(
                      color: const Color(0xFF102015),
                      border: Border.all(
                        color: const Color(0xFF10B981),
                        width: 2,
                      ),
                      borderRadius: BorderRadius.circular(20),
                    ),
                    child: LayoutBuilder(
                      builder: (context, constraints) {
                        return Stack(
                          children: [
                            Positioned.fill(
                              child: _capturedImageBytes != null
                                  ? Image.memory(
                                      _capturedImageBytes!,
```

```
                            fit: BoxFit.cover,
                          )
                        : const Center(
                            child: CircularProgressIndicator(
                              color: Color(0xFF10B981),
                            ),
                          ),
                    ),
                    AnimatedBuilder(
                      animation: _scannerAnimation,
                      builder: (context, child) {
                        final maxTop = (constraints.maxHeight - 4)
                            .clamp(0.0, double.infinity);
                        return Positioned(
                          top: _scannerAnimation.value * maxTop,
                          left: 0,
                          right: 0,
                          child: Container(
                            height: 4,
                            decoration: BoxDecoration(
                              color: const Color(0xFF10B981),
                              boxShadow: [
                                BoxShadow(
                                  color: const Color(0xFF10B981)
                                      .withValues(alpha: 0.8),
                                  blurRadius: 10,
                                  spreadRadius: 3,
                                ),
                              ],
                            ),
                          ),
                        );
                      },
                    ),
                  ],
                );
              },
            ),
          ),
        ),
      ),
      const SizedBox(height: 36),
      Text(
        _scanningStatus,
        textAlign: TextAlign.center,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 16,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 16),
      Padding(
        padding: const EdgeInsets.symmetric(horizontal: 40),
        child: Column(
          children: [
            ClipRRect(
              borderRadius: BorderRadius.circular(8),
              child: LinearProgressIndicator(
                value: _scanningProgress,
                backgroundColor: const Color(0xFF1B3524),
                valueColor: const AlwaysStoppedAnimation<Color>(
                  Color(0xFF10B981),
                ),
                minHeight: 6,
              ),
            ),
            const SizedBox(height: 8),
            Text(
              '${(_scanningProgress * 100).toInt()}% Selesai',
              style: const TextStyle(
                color: Color(0xFF10B981),
                fontSize: 13,
```

```
                                fontWeight: FontWeight.bold,
                              ),
                            ),
                          ],
                        ),
                      ),
                    ],
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }

  Widget _buildCornerMarkers() {
    const double markerSize = 24.0;
    const double borderThickness = 3.0;
    const Color markerColor = Color(0xFF10B981);

    return Positioned.fill(
      child: Padding(
        padding: const EdgeInsets.all(14.0),
        child: Stack(
          children: [
            Positioned(
              top: 0,
              left: 0,
              child: Container(
                width: markerSize,
                height: markerSize,
                decoration: const BoxDecoration(
                  border: Border(
                    top: BorderSide(color: markerColor, width: borderThickness),
                    left: BorderSide(color: markerColor, width: borderThickness),
                  ),
                ),
              ),
            ),
            Positioned(
              top: 0,
              right: 0,
              child: Container(
                width: markerSize,
                height: markerSize,
                decoration: const BoxDecoration(
                  border: Border(
                    top: BorderSide(color: markerColor, width: borderThickness),
                    right: BorderSide(color: markerColor, width: borderThickness),
                  ),
                ),
              ),
            ),
            Positioned(
              bottom: 0,
              left: 0,
              child: Container(
                width: markerSize,
                height: markerSize,
                decoration: const BoxDecoration(
                  border: Border(
                    bottom: BorderSide(color: markerColor, width: borderThickness),
                    left: BorderSide(color: markerColor, width: borderThickness),
                  ),
                ),
              ),
            ),
            Positioned(
              bottom: 0,
              right: 0,
              child: Container(
```

```
                width: markerSize,
                height: markerSize,
                decoration: const BoxDecoration(
                  border: Border(
                    bottom: BorderSide(color: markerColor, width: borderThickness),
                    right: BorderSide(color: markerColor, width: borderThickness),
                  ),
                ),
              ),
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```

File 6

Halaman Hasil Validasi & Diagnosis (ResultPage) — `lib/pages/result_page.dart`

> Menampilkan pratinjau gambar hasil scan beserta 3 kondisi keluaran: penolakan objek bukan pelepah sawit (`_isNotPalmFrond`), status tanaman sawit sehat, atau kartu diagnosis penyakit utama & kandidat lain, metrik warna visual, saran penanganan, dan tombol lanjutan ke kuisioner manual.

lib/pages/result_page.dart — **Lengkap: Baris 1–804 (804 baris)**

```
import 'dart:typed_data';
import 'package:flutter/material.dart';
import '../utils/forward_chaining.dart';
import '../utils/palm_image_analyzer.dart';
import 'diagnosis_page.dart';
import 'scan_page.dart';

class ResultPage extends StatelessWidget {
  final List<String> selectedSymptomIds;
  final List<DiagnosisResult> results;
  final String? imagePath;
  final Uint8List? imageBytes;
  final PalmImageAnalysis? visualAnalysis;

  const ResultPage({
    super.key,
    required this.selectedSymptomIds,
    required this.results,
    this.imagePath,
    this.imageBytes,
    this.visualAnalysis,
  });

  bool get _isNotPalmFrond =>
      visualAnalysis != null && !visualAnalysis!.isPalmFrond;

  @override
  Widget build(BuildContext context) {
    final primaryResult = results.isNotEmpty ? results.first : null;
    final otherResults =
        results.length > 1 ? results.sublist(1) : <DiagnosisResult>[];

    return Scaffold(
      backgroundColor: const Color(0xFF0C1910),
      appBar: AppBar(
        backgroundColor: const Color(0xFF0C1910),
        elevation: 0,
        leading: IconButton(
          icon: const Icon(Icons.close, color: Colors.white),
          onPressed: () {
            Navigator.popUntil(context, (route) => route.isFirst);
          },
        ),
        title: Text(
          _isNotPalmFrond ? 'Validasi Objek Scan' : 'Hasil Diagnosis',
          style: const TextStyle(
            color: Colors.white,
            fontSize: 18,
            fontWeight: FontWeight.bold,
          ),
        ),
        centerTitle: true,
      ),
      body: SafeArea(
        child: SingleChildScrollView(
          child: Padding(
            padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 16.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                if (imageBytes != null || imagePath != null) ...[
```

```
                _buildCapturedImagePreview(),
                const SizedBox(height: 20),
              ],

              if (_isNotPalmFrond) ...[
                // Kondisi 1: Objek bukan pelepah/daun sawit -> Jangan deteksi penyakit sawit
                _buildNotPalmFrondCard(context, visualAnalysis!),
                const SizedBox(height: 20),
                _buildScanTipsCard(),
                const SizedBox(height: 24),
              ] else if (primaryResult == null) ...[
                // Kondisi 2: Pelepah sawit sehat / tidak ada gejala terdeteksi
                _buildNoDiseaseCard(context),
                if (visualAnalysis != null) ...[
                  const SizedBox(height: 20),
                  _buildVisualMetricsSection(visualAnalysis!),
                ],
                const SizedBox(height: 24),
              ] else ...[
                // Kondisi 3: Penyakit sawit terdeteksi
                _buildPrimaryDiagnosisCard(context, primaryResult),
                const SizedBox(height: 20),

                if (visualAnalysis != null) ...[
                  _buildVisualMetricsSection(visualAnalysis!),
                  const SizedBox(height: 20),
                ],

                _buildMatchedSymptomsSection(primaryResult),
                const SizedBox(height: 20),

                _buildSolutionSection(primaryResult),
                const SizedBox(height: 24),

                if (otherResults.isNotEmpty) ...[
                  const Text(
                    'Kemungkinan Penyakit Lain',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 16,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  const SizedBox(height: 12),
                  ...otherResults.map(_buildAlternativeDiagnosisItem),
                  const SizedBox(height: 24),
                ],
              ],

              // Tombol Lengkapi Gejala via Kuisioner (jika datang dari scan kamera sawit)
              if (!_isNotPalmFrond && visualAnalysis != null) ...[
                OutlinedButton.icon(
                  onPressed: () {
                    Navigator.pushReplacement(
                      context,
                      MaterialPageRoute(
                        builder: (context) => DiagnosisPage(
                          initialSymptomIds: selectedSymptomIds,
                        ),
                      ),
                    );
                  },
                  icon: const Icon(Icons.assignment_outlined, color: Color(0xFF10B981)),
                  label: const Text(
                    'Lengkapi Gejala Lain (Kuisioner)',
                    style: TextStyle(
                      color: Colors.white,
                      fontSize: 14,
                      fontWeight: FontWeight.w600,
                    ),
                  ),
                  style: OutlinedButton.styleFrom(
                    side: const BorderSide(color: Color(0xFF10B981), width: 1.5),
```

```
                    minimumSize: const Size(double.infinity, 50),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 12),
              ],

              // Tombol Scan Ulang jika berasal dari halaman scan
              if (visualAnalysis != null || imageBytes != null) ...[
                ElevatedButton.icon(
                  onPressed: () {
                    Navigator.pushReplacement(
                      context,
                      MaterialPageRoute(
                        builder: (context) => const ScanPage(),
                      ),
                    );
                  },
                  icon: const Icon(Icons.photo_camera_outlined),
                  label: Text(
                    _isNotPalmFrond ? 'Scan Ulang Pelepah Sawit' : 'Scan Pelepah Lain',
                    style: const TextStyle(fontSize: 15, fontWeight: FontWeight.bold),
                  ),
                  style: ElevatedButton.styleFrom(
                    backgroundColor: const Color(0xFF10B981),
                    foregroundColor: Colors.white,
                    minimumSize: const Size(double.infinity, 52),
                    shape: RoundedRectangleBorder(
                      borderRadius: BorderRadius.circular(12),
                    ),
                  ),
                ),
                const SizedBox(height: 12),
              ],

              // Tombol Kembali ke Beranda
              ElevatedButton(
                onPressed: () {
                  Navigator.popUntil(context, (route) => route.isFirst);
                },
                style: ElevatedButton.styleFrom(
                  backgroundColor: (visualAnalysis != null || imageBytes != null)
                      ? const Color(0xFF152A1C)
                      : const Color(0xFF10B981),
                  foregroundColor: Colors.white,
                  minimumSize: const Size(double.infinity, 52),
                  side: (visualAnalysis != null || imageBytes != null)
                      ? const BorderSide(color: Color(0xFF223E2A), width: 1.5)
                      : BorderSide.none,
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(12),
                  ),
                ),
                child: const Text(
                  'Kembali ke Beranda',
                  style: TextStyle(fontSize: 15, fontWeight: FontWeight.bold),
                ),
              ),
              const SizedBox(height: 40),
            ],
          ),
        ),
      ),
    ),
  );
}

Widget _buildCapturedImagePreview() {
  Widget imageWidget;
  if (imageBytes != null) {
    imageWidget = Image.memory(imageBytes!, fit: BoxFit.cover);
```

```
    } else if (imagePath != null && imagePath!.startsWith('assets/')) {
      imageWidget = Image.asset(imagePath!, fit: BoxFit.cover);
    } else if (imagePath != null && imagePath!.startsWith('http')) {
      imageWidget = Image.network(imagePath!, fit: BoxFit.cover);
    } else {
      imageWidget = const Center(
        child: Icon(Icons.image_not_supported_outlined, color: Colors.grey, size: 48),
      );
    }

    return ClipRRect(
      borderRadius: BorderRadius.circular(16),
      child: Container(
        width: double.infinity,
        height: 210,
        decoration: BoxDecoration(
          border: Border.all(
            color: _isNotPalmFrond ? Colors.amber : const Color(0xFF223E2A),
            width: 1.5,
          ),
          borderRadius: BorderRadius.circular(16),
        ),
        child: imageWidget,
      ),
    );
  }

  Widget _buildNotPalmFrondCard(BuildContext context, PalmImageAnalysis analysis) {
    final greenPct = (analysis.greenLeafRatio * 100).clamp(0.0, 100.0);
    final nonPlantPct = (analysis.skinOrNonPlantRatio * 100).clamp(0.0, 100.0);
    final brightnessPct = (analysis.meanBrightness * 100).clamp(0.0, 100.0);

    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(24),
      decoration: BoxDecoration(
        color: const Color(0xFF152A1C),
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: Colors.amber.withValues(alpha: 0.7), width: 1.5),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: 0.3),
            blurRadius: 12,
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: Column(
        children: [
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 5),
            decoration: BoxDecoration(
              color: Colors.amber.withValues(alpha: 0.15),
              borderRadius: BorderRadius.circular(8),
              border: Border.all(color: Colors.amber, width: 1),
            ),
            child: const Text(
              'OBJEK BUKAN PELEPAH SAWIT',
              style: TextStyle(
                color: Colors.amber,
                fontSize: 11,
                fontWeight: FontWeight.bold,
                letterSpacing: 1,
              ),
            ),
          ),
        ),
        const SizedBox(height: 20),
        Container(
          padding: const EdgeInsets.all(16),
          decoration: BoxDecoration(
            color: Colors.amber.withValues(alpha: 0.12),
            shape: BoxShape.circle,
          ),
```

```
            child: const Icon(
              Icons.warning_amber_rounded,
              color: Colors.amber,
              size: 52,
            ),
          ),
          const SizedBox(height: 16),
          Text(
            analysis.statusTitle,
            textAlign: TextAlign.center,
            style: const TextStyle(
              color: Colors.white,
              fontSize: 20,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 10),
          Text(
            analysis.statusDetail,
            textAlign: TextAlign.center,
            style: TextStyle(
              color: Colors.grey[300],
              fontSize: 14,
              height: 1.5,
            ),
          ),
          const SizedBox(height: 20),
          const Divider(color: Color(0xFF223E2A)),
          const SizedBox(height: 12),
          Row(
            children: [
              Expanded(
                child: _buildMetricChip(
                  label: 'Hijau Daun',
                  value: '${greenPct.toStringAsFixed(0)}%',
                  color: const Color(0xFF10B981),
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: _buildMetricChip(
                  label: 'Non-Sawit/Kulit',
                  value: '${nonPlantPct.toStringAsFixed(0)}%',
                  color: Colors.amber,
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: _buildMetricChip(
                  label: 'Kecerahan',
                  value: '${brightnessPct.toStringAsFixed(0)}%',
                  color: Colors.lightBlueAccent,
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }

  Widget _buildScanTipsCard() {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(18),
      decoration: BoxDecoration(
        color: const Color(0xFF122217),
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: const Color(0xFF1D3525), width: 1),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
```

```
          const Row(
            children: [
              Icon(Icons.lightbulb_outline, color: Color(0xFF10B981), size: 20),
              SizedBox(width: 8),
              Text(
                'Panduan Pengambilan Gambar Pelepah:',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 14,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ),
          const SizedBox(height: 10),
          Text(
            '1. Arahkan kamera langsung ke helai daun atau pelepah kelapa sawit (jarak ±30–60 cm).\n'
            '2. Pastikan pelepah mengisi bagian tengah bingkai kotak hijau.\n'
            '3. Hindari mengarahkan kamera ke wajah, tembok, lantai, atau objek non-tanaman.\n'
            '4. Gunakan pencahayaan alami yang cukup dan tidak terlalu silau.',
            style: TextStyle(
              color: Colors.grey[300],
              fontSize: 13,
              height: 1.5,
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildVisualMetricsSection(PalmImageAnalysis analysis) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(18),
      decoration: BoxDecoration(
        color: const Color(0xFF122217),
        borderRadius: BorderRadius.circular(16),
        border: Border.all(color: const Color(0xFF1D3525), width: 1),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Row(
            children: [
              Icon(Icons.document_scanner_outlined, color: Color(0xFF10B981), size: 18),
              SizedBox(width: 8),
              Text(
                'Analisis Visual Citra Pelepah:',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 14,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ),
          const SizedBox(height: 8),
          Text(
            analysis.statusDetail,
            style: TextStyle(color: Colors.grey[350], fontSize: 12, height: 1.4),
          ),
          const SizedBox(height: 14),
          Row(
            children: [
              Expanded(
                child: _buildMetricChip(
                  label: 'Hijau Sehat',
                  value: '${analysis.healthyVegetationPercent.toStringAsFixed(0)}%',
                  color: const Color(0xFF10B981),
                ),
              ),
              const SizedBox(width: 8),
```

```
              Expanded(
                child: _buildMetricChip(
                  label: 'Menguning',
                  value: '${analysis.yellowingPercent.toStringAsFixed(0)}%',
                  color: Colors.amber,
                ),
              ),
              const SizedBox(width: 8),
              Expanded(
                child: _buildMetricChip(
                  label: 'Bercak/Busuk',
                  value: '${analysis.lesionPercent.toStringAsFixed(0)}%',
                  color: const Color(0xFFEF4444),
                ),
              ),
            ],
          ),
        ],
      ),
    );
  }

  Widget _buildMetricChip({
    required String label,
    required String value,
    required Color color,
  }) {
    return Container(
      padding: const EdgeInsets.symmetric(vertical: 10, horizontal: 8),
      decoration: BoxDecoration(
        color: color.withValues(alpha: 0.1),
        borderRadius: BorderRadius.circular(10),
        border: Border.all(color: color.withValues(alpha: 0.35), width: 1),
      ),
      child: Column(
        children: [
          Text(
            value,
            style: TextStyle(
              color: color,
              fontSize: 16,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 2),
          Text(
            label,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
            style: TextStyle(
              color: Colors.grey[300],
              fontSize: 11,
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildNoDiseaseCard(BuildContext context) {
    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(28),
      decoration: BoxDecoration(
        color: const Color(0xFF152A1C),
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: const Color(0xFF223E2A), width: 1.5),
      ),
      child: Column(
        children: [
          Container(
            padding: const EdgeInsets.all(16),
            decoration: BoxDecoration(
```

```
              color: Colors.green.withValues(alpha: 0.1),
              shape: BoxShape.circle,
            ),
            child: const Icon(
              Icons.check_circle_outline,
              color: Colors.green,
              size: 52,
            ),
          ),
          const SizedBox(height: 20),
          const Text(
            'Tanaman Sawit Sehat',
            style: TextStyle(
              color: Colors.white,
              fontSize: 20,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 12),
          Text(
            visualAnalysis != null
                ? 'Pelepah sawit terdeteksi dalam kondisi dominan hijau segar tanpa indikasi gejala penyakit yang berarti.'
                : 'Berdasarkan gejala-gejala yang Anda masukkan, sistem tidak menemukan indikasi penyakit kelapa sawit yang
cocok dengan database kami.',
            textAlign: TextAlign.center,
            style: TextStyle(
              color: Colors.grey[300],
              fontSize: 14,
              height: 1.5,
            ),
          ),
          const SizedBox(height: 20),
          const Divider(color: Color(0xFF223E2A)),
          const SizedBox(height: 16),
          const Text(
            'Rekomendasi Pemeliharaan:',
            style: TextStyle(
              color: Color(0xFF10B981),
              fontSize: 13,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 8),
          Text(
            'Tetap lakukan pemantauan berkala, pemupukan berimbang NPK sesuai dosis, serta pembersihan gulma di sekitar
piringan pohon sawit secara terjadwal.',
            textAlign: TextAlign.center,
            style: TextStyle(
              color: Colors.grey[400],
              fontSize: 13,
              height: 1.4,
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildPrimaryDiagnosisCard(BuildContext context, DiagnosisResult result) {
    final isHighConfidence = result.confidence >= 75.0;
    final statusColor = isHighConfidence ? const Color(0xFFEF4444) : Colors.orangeAccent;

    return Container(
      width: double.infinity,
      padding: const EdgeInsets.all(24),
      decoration: BoxDecoration(
        color: const Color(0xFF152A1C),
        borderRadius: BorderRadius.circular(20),
        border: Border.all(color: statusColor.withValues(alpha: 0.5), width: 1.5),
        boxShadow: [
          BoxShadow(
            color: Colors.black.withValues(alpha: 0.3),
            blurRadius: 12,
```

```
            offset: const Offset(0, 6),
          ),
        ],
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceBetween,
            children: [
              Container(
                padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
                decoration: BoxDecoration(
                  color: statusColor.withValues(alpha: 0.15),
                  borderRadius: BorderRadius.circular(8),
                  border: Border.all(color: statusColor, width: 1),
                ),
                child: Text(
                  isHighConfidence ? 'TERDETEKSI KUAT' : 'INDIKASI GEJALA',
                  style: TextStyle(
                    color: statusColor,
                    fontSize: 10,
                    fontWeight: FontWeight.bold,
                    letterSpacing: 1,
                  ),
                ),
              ),
              Row(
                children: [
                  const Icon(Icons.analytics_outlined, color: Color(0xFF10B981), size: 18),
                  const SizedBox(width: 4),
                  Text(
                    '${result.confidence.toStringAsFixed(1)}% Cocok',
                    style: const TextStyle(
                      color: Color(0xFF10B981),
                      fontSize: 14,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ],
              ),
            ],
          ),
          const SizedBox(height: 20),
          Text(
            result.disease.name,
            style: const TextStyle(
              color: Colors.white,
              fontSize: 22,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 6),
          Text(
            'Kode Penyakit: ${result.disease.id}',
            style: TextStyle(
              color: Colors.grey[500],
              fontSize: 13,
            ),
          ),
          const SizedBox(height: 16),
          const Text(
            'Deskripsi Penyakit:',
            style: TextStyle(
              color: Color(0xFF10B981),
              fontSize: 13,
              fontWeight: FontWeight.bold,
            ),
          ),
          const SizedBox(height: 6),
          Text(
            result.disease.description,
            style: TextStyle(
```

```
            color: Colors.grey[300],
            fontSize: 13,
            height: 1.5,
          ),
        ),
      ],
    ),
  );
}

Widget _buildMatchedSymptomsSection(DiagnosisResult result) {
  return Container(
    width: double.infinity,
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: const Color(0xFF122217),
      borderRadius: BorderRadius.circular(16),
      border: Border.all(color: const Color(0xFF1D3525), width: 1),
    ),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        const Text(
          'Gejala yang Terpenuhi:',
          style: TextStyle(
            color: Colors.white,
            fontSize: 14,
            fontWeight: FontWeight.bold,
          ),
        ),
        const SizedBox(height: 12),
        ...result.matchedSymptoms.map((sId) {
          final symptom = ForwardChainingEngine.symptoms.firstWhere(
            (s) => s.id == sId,
            orElse: () => Symptom(id: sId, name: sId, question: ''),
          );
          return Padding(
            padding: const EdgeInsets.only(bottom: 8.0),
            child: Row(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                const Icon(
                  Icons.check_box_outlined,
                  color: Color(0xFF10B981),
                  size: 18,
                ),
                const SizedBox(width: 8),
                Expanded(
                  child: Text(
                    '[$sId] ${symptom.name}',
                    style: TextStyle(
                      color: Colors.grey[300],
                      fontSize: 13,
                      height: 1.3,
                    ),
                  ),
                ),
              ],
            ),
          );
        }),
      ],
    ),
  );
}

Widget _buildSolutionSection(DiagnosisResult result) {
  return Container(
    width: double.infinity,
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: const Color(0xFF152A1C),
      borderRadius: BorderRadius.circular(16),
```

```
        border: Border.all(color: const Color(0xFF223E2A), width: 1),
      ),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Row(
            children: [
              Icon(Icons.healing_outlined, color: Colors.orangeAccent, size: 20),
              SizedBox(width: 8),
              Text(
                'Saran Penanganan:',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 15,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Text(
            result.disease.solution,
            style: TextStyle(
              color: Colors.grey[300],
              fontSize: 13,
              height: 1.5,
            ),
          ),
        ],
      ),
    );
  }

  Widget _buildAlternativeDiagnosisItem(DiagnosisResult result) {
    return Container(
      margin: const EdgeInsets.only(bottom: 8),
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
      decoration: BoxDecoration(
        color: const Color(0xFF152A1C).withValues(alpha: 0.6),
        borderRadius: BorderRadius.circular(12),
        border: Border.all(color: const Color(0xFF223E2A), width: 1),
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceBetween,
        children: [
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  result.disease.name,
                  style: const TextStyle(
                    color: Colors.white,
                    fontSize: 14,
                    fontWeight: FontWeight.bold,
                  ),
                ),
                const SizedBox(height: 4),
                Text(
                  'Cocok ${result.matchedSymptoms.length} dari ${result.totalSymptoms} gejala',
                  style: TextStyle(
                    color: Colors.grey[400],
                    fontSize: 12,
                  ),
                ),
              ],
            ),
          ),
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
            decoration: BoxDecoration(
              color: const Color(0xFF10B981).withValues(alpha: 0.1),
              borderRadius: BorderRadius.circular(6),
```

```
            ),
            child: Text(
              '${result.confidence.toStringAsFixed(0)}%',
              style: const TextStyle(
                color: Color(0xFF10B981),
                fontSize: 13,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ],
      ),
    );
  }
}
```

File 7

Halaman Kuisioner Gejala Manual (DiagnosisPage) — `lib/pages/diagnosis_page.dart`

> Halaman konsultasi gejala satu per satu (`G01` – `G13`) yang mendukung pengisian awal otomatis dari hasil scan foto (`initialSymptomIds`), navigasi maju/mundur beserta indikator jawaban sebelumnya, dan tombol *Selesai Lebih Cepat*.

lib/pages/diagnosis_page.dart — **Lengkap: Baris 1–327 (327 baris)**

```
import 'package:flutter/material.dart';
import '../utils/forward_chaining.dart';
import 'result_page.dart';

class DiagnosisPage extends StatefulWidget {
  final List<String>? initialSymptomIds;

  const DiagnosisPage({super.key, this.initialSymptomIds});

  @override
  State<DiagnosisPage> createState() => _DiagnosisPageState();
}

class _DiagnosisPageState extends State<DiagnosisPage> {
  // Map untuk menyimpan jawaban gejala (key: symptomId, value: isSelected)
  final Map<String, bool> _answers = {};
  final Set<String> _answeredIds = {};
  int _currentIndex = 0;

  @override
  void initState() {
    super.initState();
    final initialSet = widget.initialSymptomIds?.toSet() ?? const <String>{};
    // Inisialisasi jawaban gejala (pre-fill jika berasal dari hasil scan kamera)
    for (var symptom in ForwardChainingEngine.symptoms) {
      final hasInitial = initialSet.contains(symptom.id);
      _answers[symptom.id] = hasInitial;
      if (hasInitial) {
        _answeredIds.add(symptom.id);
      }
    }
  }

  void _nextQuestion() {
    if (_currentIndex < ForwardChainingEngine.symptoms.length - 1) {
      setState(() {
        _currentIndex++;
      });
    } else {
      _finishDiagnosis();
    }
  }

  void _previousQuestion() {
    if (_currentIndex > 0) {
      setState(() {
        _currentIndex--;
      });
    }
  }

  void _finishDiagnosis() {
    // Kumpulkan ID gejala yang dijawab 'Ya' (true)
    List<String> selectedSymptomIds = [];
    _answers.forEach((id, val) {
      if (val == true) {
        selectedSymptomIds.add(id);
      }
    });

    // Jalankan Forward Chaining Engine
    List<DiagnosisResult> diagnosisResults = ForwardChainingEngine.diagnose(selectedSymptomIds);
```

```
    // Pindah ke Halaman Hasil
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (context) => ResultPage(
          selectedSymptomIds: selectedSymptomIds,
          results: diagnosisResults,
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final symptoms = ForwardChainingEngine.symptoms;
    final currentSymptom = symptoms[_currentIndex];
    final progress = (_currentIndex + 1) / symptoms.length;

    return Scaffold(
      backgroundColor: const Color(0xFF0C1910),
      appBar: AppBar(
        backgroundColor: const Color(0xFF0C1910),
        elevation: 0,
        leading: IconButton(
          icon: const Icon(Icons.arrow_back, color: Colors.white),
          onPressed: () => Navigator.pop(context),
        ),
        title: const Text(
          'Konsultasi Gejala',
          style: TextStyle(color: Colors.white, fontSize: 18, fontWeight: FontWeight.bold),
        ),
        centerTitle: true,
      ),
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 24.0, vertical: 16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              // Progress Bar
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceBetween,
                children: [
                  Text(
                    'Pertanyaan ${_currentIndex + 1} dari ${symptoms.length}',
                    style: TextStyle(
                      color: Colors.grey[400],
                      fontSize: 13,
                      fontWeight: FontWeight.w500,
                    ),
                  ),
                  Text(
                    '${((_currentIndex + 1) / symptoms.length * 100).toInt()}%',
                    style: const TextStyle(
                      color: Color(0xFF10B981),
                      fontSize: 13,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ],
              ),
              const SizedBox(height: 10),
              ClipRRect(
                borderRadius: BorderRadius.circular(8),
                child: LinearProgressIndicator(
                  value: progress,
                  backgroundColor: const Color(0xFF1B3524),
                  valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFF10B981)),
                  minHeight: 8,
                ),
              ),
              const SizedBox(height: 40),
```

```
// Question Card
Expanded(
  child: Container(
    width: double.infinity,
    padding: const EdgeInsets.all(24),
    decoration: BoxDecoration(
      color: const Color(0xFF152A1C),
      borderRadius: BorderRadius.circular(20),
      border: Border.all(color: const Color(0xFF223E2A), width: 1.5),
      boxShadow: [
        BoxShadow(
          color: Colors.black.withValues(alpha: 0.2),
          blurRadius: 10,
          offset: const Offset(0, 5),
        ),
      ],
    ),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        // Icon/Badge Gejala
        Container(
          padding: const EdgeInsets.all(16),
          decoration: BoxDecoration(
            color: const Color(0xFF10B981).withValues(alpha: 0.1),
            shape: BoxShape.circle,
          ),
          child: const Icon(
            Icons.help_outline,
            color: Color(0xFF10B981),
            size: 40,
          ),
        ),
        const SizedBox(height: 24),
        Text(
          currentSymptom.id,
          style: const TextStyle(
            color: Color(0xFF10B981),
            fontSize: 14,
            fontWeight: FontWeight.bold,
            letterSpacing: 1.5,
          ),
        ),
        const SizedBox(height: 16),
        Text(
          currentSymptom.question,
          textAlign: TextAlign.center,
          style: const TextStyle(
            color: Colors.white,
            fontSize: 18,
            fontWeight: FontWeight.w600,
            height: 1.5,
          ),
        ),
        if (_answeredIds.contains(currentSymptom.id)) ...[
          const SizedBox(height: 16),
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
            decoration: BoxDecoration(
              color: (_answers[currentSymptom.id] == true
                      ? const Color(0xFF10B981)
                      : const Color(0xFFEF4444))
                  .withValues(alpha: 0.15),
              borderRadius: BorderRadius.circular(20),
              border: Border.all(
                color: _answers[currentSymptom.id] == true
                    ? const Color(0xFF10B981)
                    : const Color(0xFFEF4444),
              ),
            ),
            child: Text(
              _answers[currentSymptom.id] == true
```

```
                      ? 'Jawaban saat ini: YA'
                      : 'Jawaban saat ini: TIDAK',
                  style: TextStyle(
                    color: _answers[currentSymptom.id] == true
                        ? const Color(0xFF34D399)
                        : const Color(0xFFEF4444),
                    fontSize: 12,
                    fontWeight: FontWeight.w600,
                  ),
                ),
              ),
            ],
          ],
        ),
      ),
    ),
    const SizedBox(height: 30),

    // Action Buttons
    Row(
      children: [
        // Button TIDAK
        Expanded(
          child: OutlinedButton(
            onPressed: () {
              setState(() {
                _answers[currentSymptom.id] = false;
                _answeredIds.add(currentSymptom.id);
              });
              _nextQuestion();
            },
            style: OutlinedButton.styleFrom(
              side: const BorderSide(color: Color(0xFFEF4444), width: 1.5),
              padding: const EdgeInsets.symmetric(vertical: 16),
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(12),
              ),
            ),
            child: const Text(
              'TIDAK',
              style: TextStyle(
                color: Color(0xFFEF4444),
                fontSize: 16,
                fontWeight: FontWeight.bold,
                letterSpacing: 1,
              ),
            ),
          ),
        ),
        const SizedBox(width: 16),

        // Button YA
        Expanded(
          child: ElevatedButton(
            onPressed: () {
              setState(() {
                _answers[currentSymptom.id] = true;
                _answeredIds.add(currentSymptom.id);
              });
              _nextQuestion();
            },
            style: ElevatedButton.styleFrom(
              backgroundColor: const Color(0xFF10B981),
              foregroundColor: Colors.white,
              padding: const EdgeInsets.symmetric(vertical: 16),
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(12),
              ),
              elevation: 3,
              shadowColor: const Color(0xFF10B981).withValues(alpha: 0.3),
            ),
            child: const Text(
              'YA',
```

```
                        style: TextStyle(
                          fontSize: 16,
                          fontWeight: FontWeight.bold,
                          letterSpacing: 1,
                        ),
                      ),
                    ),
                  ),
                ),
              ],
            ),
            const SizedBox(height: 20),

            // Navigation Bottom Row
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceBetween,
              children: [
                // Back Button (only shown if not on the first question)
                _currentIndex > 0
                    ? TextButton.icon(
                        onPressed: _previousQuestion,
                        icon: const Icon(Icons.arrow_back_ios, size: 14, color: Colors.grey),
                        label: const Text(
                          'Kembali',
                          style: TextStyle(color: Colors.grey, fontSize: 14),
                        ),
                      )
                    : const SizedBox.shrink(),

                // Skip / Finish early button
                TextButton.icon(
                  onPressed: _finishDiagnosis,
                  icon: const Icon(Icons.check_circle_outline, size: 16, color: Color(0xFF10B981)),
                  label: const Text(
                    'Selesai Lebih Cepat',
                    style: TextStyle(color: Color(0xFF10B981), fontSize: 14, fontWeight: FontWeight.bold),
                  ),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

File 8

Unit Test Validasi Citra Pelepah Sawit — `test/palm_image_analyzer_test.dart`

> Pengujian terotomasi menggunakan citra sintetis RGBA untuk memastikan wajah/kulit, latar tembok abu-abu, frame terlalu gelap, dan bidang hijau polos ditolak, sedangkan pola pelepah hijau sehat, menguning, dan bercak antraknosa terdeteksi sesuai ekspektasi.

test/palm_image_analyzer_test.dart — **Lengkap: Baris 1–139 (139 baris)**

```
import 'dart:typed_data';
import 'package:flutter_test/flutter_test.dart';
import 'package:sawit_detector/utils/palm_image_analyzer.dart';

Uint8List _createRgbaGrid(
  int width,
  int height,
  List<int> Function(int x, int y) pixelBuilder,
) {
  final buffer = Uint8List(width * height * 4);
  for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
      final rgb = pixelBuilder(x, y);
      final idx = (y * width + x) * 4;
      buffer[idx] = rgb[0];
      buffer[idx + 1] = rgb[1];
      buffer[idx + 2] = rgb[2];
      buffer[idx + 3] = 255;
    }
  }
  return buffer;
}

void main() {
  group('Pengujian PalmImageAnalyzer (Validasi Pelepah Sawit & Deteksi Visual)', () {
    test('Harus menolak gambar wajah / warna kulit sebagai bukan pelepah sawit', () {
      final rgba = _createRgbaGrid(64, 64, (x, y) {
        // Warna kulit manusia dengan sedikit variasi pencahayaan
        final delta = (x % 3) * 4;
        return [210 - delta, 155 - delta, 125 - delta];
      });

      final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);

      expect(result.isPalmFrond, false);
      expect(result.validationError, PalmValidationError.nonPlantSkinOrFace);
      expect(result.detectedSymptomIds, isEmpty);
    });

    test('Harus menolak wajah di tengah bingkai meskipun ada latar hijau di pinggir', () {
      final rgba = _createRgbaGrid(64, 64, (x, y) {
        final inCenter = x >= 14 && x <= 50 && y >= 14 && y <= 50;
        if (inCenter) {
          return [205, 148, 118]; // Wajah di tengah
        }
        return (x + y) % 2 == 0 ? [45, 130, 50] : [60, 150, 65]; // Daun di tepi
      });

      final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);

      expect(result.isPalmFrond, false);
      expect(result.validationError, PalmValidationError.nonPlantSkinOrFace);
      expect(result.detectedSymptomIds, isEmpty);
    });

    test('Harus menolak tembok polos / latar abu-abu netral', () {
      final rgba = _createRgbaGrid(64, 64, (x, y) => [215, 215, 218]);

      final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);

      expect(result.isPalmFrond, false);
```

```
    expect(result.validationError, PalmValidationError.neutralBackground);
    expect(result.detectedSymptomIds, isEmpty);
  });

  test('Harus menolak gambar yang terlalu gelap', () {
    final rgba = _createRgbaGrid(64, 64, (x, y) => [10, 12, 10]);

    final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);

    expect(result.isPalmFrond, false);
    expect(result.validationError, PalmValidationError.tooDark);
  });

  test('Harus menolak bidang hijau polos tanpa tekstur helai daun', () {
    final rgba = _createRgbaGrid(64, 64, (x, y) => [40, 150, 45]);

    final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);

    expect(result.isPalmFrond, false);
    expect(result.validationError, PalmValidationError.flatGreenSurface);
  });

  test('Harus mengenali pelepah sawit hijau bertekstur sebagai sehat', () {
    final rgba = _createRgbaGrid(64, 64, (x, y) {
      // Pola garis lidi/helai anak daun hijau sawit
      if (x % 4 == 0) {
        return [30, 95, 35]; // Bayangan celah helai daun
      }
      return [55, 145, 50]; // Hijau daun segar
    });

    final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);

    expect(result.isPalmFrond, true);
    expect(result.detectedSymptomIds, isEmpty);
    expect(result.healthyVegetationPercent, greaterThan(75.0));
  });

  test('Harus mendeteksi gejala menguning luas (G01) pada pelepah sawit', () {
    final rgba = _createRgbaGrid(64, 64, (x, y) {
      if (y < 24) {
        return x % 3 == 0 ? [40, 110, 35] : [60, 145, 55]; // Sebagian hijau
      }
      // Area pelepah menguning klorosis
      return y % 4 == 0 ? [185, 170, 35] : [205, 190, 45];
    });

    final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);

    expect(result.isPalmFrond, true);
    expect(result.detectedSymptomIds.contains('G01'), true);
  });

  test('Harus mendeteksi gejala bercak gelap dengan halo kuning (G12)', () {
    final rgba = _createRgbaGrid(64, 64, (x, y) {
      // Buat 6 bercak coklat gelap dikelilingi lingkaran kuning di atas daun hijau
      for (final cx in [16, 32, 48]) {
        for (final cy in [20, 44]) {
          final dx = (x - cx).abs();
          final dy = (y - cy).abs();
          if (dx <= 1 && dy <= 1) {
            return [65, 42, 22]; // Pusat bercak coklat gelap
          }
          if (dx <= 3 && dy <= 3) {
            return [205, 190, 40]; // Halo kuning
          }
        }
      }
      return x % 5 == 0 ? [38, 105, 38] : [55, 145, 50];
    });

    final result = PalmImageAnalyzer.analyzeRgba(rgba, 64, 64);
```

```
      expect(result.isPalmFrond, true);
      expect(result.detectedSymptomIds.contains('G12'), true);
    });
  });
}
```

File 9

Unit Test Engine Forward Chaining — `test/forward_chaining_test.dart`

> Pengujian inferensi aturan `R1` – `R6`, perhitungan skor keyakinan, `totalSymptoms`, dan pengurutan kandidat penyakit.

test/forward_chaining_test.dart — Lengkap: Baris 1–66 (66 baris)

```
import 'package:flutter_test/flutter_test.dart';
import 'package:sawit_detector/utils/forward_chaining.dart';

void main() {
  group('Pengujian Engine Forward Chaining Deteksi Penyakit Sawit', () {
    test('Harus mengembalikan daftar kosong jika tidak ada gejala yang dipilih', () {
      final results = ForwardChainingEngine.diagnose([]);
      expect(results.isEmpty, true);
    });

    test('Harus mendeteksi Ganoderma (P01) dengan kecocokan 100% saat gejala G01, G02, G03, G04 terpenuhi', () {
      final selectedSymptoms = ['G01', 'G02', 'G03', 'G04'];
      final results = ForwardChainingEngine.diagnose(selectedSymptoms);

      expect(results.isNotEmpty, true);

      // Ambil hasil teratas (confidence tertinggi)
      final primaryResult = results.first;
      expect(primaryResult.disease.id, 'P01');
      expect(primaryResult.confidence, 100.0);
      expect(primaryResult.matchedSymptoms.length, 4);
      expect(primaryResult.totalSymptoms, 4);
    });

    test('Harus mendeteksi Busuk Pucuk (P02) dengan kecocokan 100% saat gejala G05, G06 terpenuhi', () {
      final selectedSymptoms = ['G05', 'G06'];
      final results = ForwardChainingEngine.diagnose(selectedSymptoms);

      expect(results.isNotEmpty, true);
      final primaryResult = results.first;
      expect(primaryResult.disease.id, 'P02');
      expect(primaryResult.confidence, 100.0);
    });

    test('Harus menghitung kecocokan parsial secara akurat jika hanya sebagian gejala terpenuhi', () {
      // Penyakit Mahkota (P03) butuh G07, G08, G09
      // Kita pilih G07 dan G09 saja (2 dari 3 gejala = 66.67%)
      final selectedSymptoms = ['G07', 'G09'];
      final results = ForwardChainingEngine.diagnose(selectedSymptoms);

      expect(results.isNotEmpty, true);

      // Cari hasil untuk penyakit P03
      final crownDiseaseResult = results.firstWhere((r) => r.disease.id == 'P03');
      expect(crownDiseaseResult.confidence, closeTo(66.67, 0.01));
      expect(crownDiseaseResult.matchedSymptoms.contains('G07'), true);
      expect(crownDiseaseResult.matchedSymptoms.contains('G09'), true);
    });

    test('Harus mengurutkan hasil berdasarkan persentase kecocokan (confidence) tertinggi', () {
      // Kita pilih G01 (bagian dari P01 - 1/4 = 25%) dan G05, G06 (bagian dari P02 - 100%)
      final selectedSymptoms = ['G01', 'G05', 'G06'];
      final results = ForwardChainingEngine.diagnose(selectedSymptoms);

      expect(results.length >= 2, true);

      // P02 harus berada di urutan pertama (100% > 25%)
      expect(results[0].disease.id, 'P02');
      expect(results[0].confidence, 100.0);

      // P01 harus berada di urutan setelahnya
      expect(results[1].disease.id, 'P01');
      expect(results[1].confidence, 25.0);
```

```
    });
  });
}
```

File 10

Konfigurasi CI/CD GitHub Actions — `.github/workflows/flutter_ci.yml`

> Workflow build otomatis pada setiap push ke branch `main` / `master` untuk menjalankan `flutter pub get`, `flutter analyze`, `flutter test`, dan menghasilkan artefak Android APK rilis.

`.github/workflows/flutter_ci.yml` — Lengkap: Baris 1–72 (72 baris)

```
name: Flutter CI & Build APK

on:
  push:
    branches:
      - main
      - master
  pull_request:
    branches:
      - main
      - master
  workflow_dispatch:

jobs:
  analyze_test_and_build:
    name: Analyze, Test & Build Android APK
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Java 17
        uses: actions/setup-java@v4
        with:
          distribution: 'temurin'
          java-version: '17'

      - name: Setup Flutter (stable)
        uses: subosito/flutter-action@v2
        with:
          channel: 'stable'
          cache: true

      - name: Install dependencies
        run: flutter pub get

      - name: Run static analysis
        run: |
          set +e
          flutter analyze > analyze.log 2>&1
          EXIT_CODE=$?
          cat analyze.log
          if [ $EXIT_CODE -ne 0 ]; then
            while IFS= read -r line; do
              [ -n "$line" ] && echo "::error::$line"
            done < analyze.log
          fi
          exit $EXIT_CODE

      - name: Run unit & widget tests
        run: |
          set +e
          flutter test -r expanded > test.log 2>&1
          EXIT_CODE=$?
          cat test.log
          if [ $EXIT_CODE -ne 0 ]; then
            grep -A 2 -E "(The following assertion was thrown:|The following message was thrown:|\[E\])" test.log | grep -v
-E "^(--|The following assertion was thrown:|The following message was thrown:)$" | head -n 10 | while IFS= read -r line;
do
              echo "::error::$line"
            done
```

```
      fi
      exit $EXIT_CODE

  - name: Build Android Release APK
    run: flutter build apk --release

  - name: Upload APK artifact
    uses: actions/upload-artifact@v4
    with:
      name: sawit-detector-release-apk
      path: build/app/outputs/flutter-apk/app-release.apk
      retention-days: 14
```

File 11

Konfigurasi Proyek & Dependensi Flutter — `pubspec.yaml`

> Definisi metadata proyek `sawit_detector`, versi Dart SDK, serta daftar dependensi (`camera`, `image_picker`, `cupertino_icons`).

pubspec.yaml — Lengkap: Baris 1–98 (98 baris)

```
name: sawit_detector
description: "A new Flutter project."
# The following line prevents the package from being accidentally published to
# pub.dev using `flutter pub publish`. This is preferred for private packages.
publish_to: 'none' # Remove this line if you wish to publish to pub.dev

# The following defines the version and build number for your application.
# A version number is three numbers separated by dots, like 1.2.43
# followed by an optional build number separated by a +.
# Both the version and the builder number may be overridden in flutter
# build by specifying --build-name and --build-number, respectively.
# In Android, build-name is used as versionName while build-number used as versionCode.
# Read more about Android versioning at https://developer.android.com/studio/publish/versioning
# In iOS, build-name is used as CFBundleShortVersionString while build-number is used as CFBundleVersion.
# Read more about iOS versioning at
#
https://developer.apple.com/library/archive/documentation/General/Reference/InfoPlistKeyReference/Articles/CoreFoundationKe
ys.html
# In Windows, build-name is used as the major, minor, and patch parts
# of the product and file versions while build-number is used as the build suffix.
version: 1.0.0+1

environment:
  sdk: ^3.11.5

# Dependencies specify other packages that your package needs in order to work.
# To automatically upgrade your package dependencies to the latest versions
# consider running `flutter pub upgrade --major-versions`. Alternatively,
# dependencies can be manually updated by changing the version numbers below to
# the latest version available on pub.dev. To see which dependencies have newer
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  camera: ^0.12.0+2
  image_picker: ^1.1.2

dev_dependencies:
  flutter_test:
    sdk: flutter

  # The "flutter_lints" package below contains a set of recommended lints to
  # encourage good coding practices. The lint set provided by the package is
  # activated in the `analysis_options.yaml` file located at the root of your
  # package. See that file for information about deactivating specific lint
  # rules and activating additional ones.
  flutter_lints: ^6.0.0
  flutter_launcher_icons: ^0.14.4

# For information on the generic Dart part of this file, see the
# following page: https://dart.dev/tools/pub/pubspec

# The following section is specific to Flutter packages.
flutter:

  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons in
  # the material Icons class.
```

```
  uses-material-design: true

  assets:
    - assets/logo.jpg

  # An image asset can refer to one or more resolution-specific "variants", see
  # https://flutter.dev/to/resolution-aware-images

  # For details regarding adding assets from package dependencies, see
  # https://flutter.dev/to/asset-from-package

  # To add custom fonts to your application, add a fonts section here,
  # in this "flutter" section. Each entry in this list should have a
  # "family" key with the font family name, and a "fonts" key with a
  # list giving the asset and other descriptors for the font. For
  # example:
  # fonts:
  #   - family: Schyler
  #     fonts:
  #       - asset: fonts/Schyler-Regular.ttf
  #       - asset: fonts/Schyler-Italic.ttf
  #         style: italic
  #   - family: Trajan Pro
  #     fonts:
  #       - asset: fonts/TrajanPro.ttf
  #       - asset: fonts/TrajanPro_Bold.ttf
  #         weight: 700
  #
  # For details regarding fonts from package dependencies,
  # see https://flutter.dev/to/font-from-package

flutter_launcher_icons:
  android: "launcher_icon"
  ios: true
  image_path: "assets/logo.jpg"
  remove_alpha_ios: true
  adaptive_icon_background: "#0C1910"
  adaptive_icon_foreground: "assets/logo_foreground.png"
```